Benchmarking Decision Models for LLM Safety Screening

Mark Torres* William J. Brady†

Abstract

Typed decision models answer fixed questions about text and return calibrated probabilities; managed guardrails return ordinal confidence bands on named content filters. We benchmark TypeSafe Jev (`jev-1.13.0`) TypeSafe AI [2026], Deußer et al. [2026b], Cloudflare Clef and Clef-flash Cloudflare [2026a,b], and Amazon Bedrock Guardrails (input content filters only) Amazon Web Services [2024], alone and in union, intersection, and cascade stacks, on eleven public prompt-level benchmarks: ToxiGen Hartvigsen et al. [2022], OpenAI Moderation Markov et al. [2023], ToxicChat Lin et al. [2023], prompt injections deepset [2024], WildGuardMix Han et al. [2024], Do-Not-Answer Wang et al. [2024], XSTest Röttger et al. [2024], HarmBench Mazeika et al. [2024], JailbreakBench Chao et al. [2024], GoEmotions Demszky et al. [2020], and Emotion Saravia et al. [2018]. Harmful prompts are described by dataset and metric only.

A harness check reproduced published Jev intervals on six overlapping sets before the full pack was scored Deußer et al. [2026b]. On ToxiGen, tuned Jev accuracy is 0.8755. ToxicChat toxic F1 is 0.7878. On Do-Not-Answer, Jev detection rate is 0.7433 (aggregates only; no question text). On XSTest safe prompts at tuned thresholds, Jev false-positive rate is 0.0120. HarmBench miss rate is 0.2026 for guardrails versus 0.2870 for Jev. On JailbreakBench, the pre-registered guardrail crosswalk yields miss rate 0.9800 versus 0.0300 for Jev; a post-hoc mapping that takes the maximum band across all content filters lowers medium-strength miss rate to 0.2300 at false-positive rate 0.1900. Median latency is 88.9 ms for Jev versus 359 ms for guardrails. Cost per 1,000 prompts is $0.023 for Jev, $0.230 for Clef, and $0.179 for guardrails; total provider spend is $18.38.

1 Introduction

Products that support mental health and emotional wellbeing must screen user input before a large language model responds. A harmful request that passes the filter can enter the session. A false positive can refuse someone who was describing ordinary distress. The screening layer therefore trades missed harm against over-refusal, latency, and cost.

Two families of commercial APIs implement that layer in different ways. Typed decision models, such as TypeSafe Jev TypeSafe AI [2026], Deußer et al. [2026b], take a text state and one or more fixed questions, then return probabilities or discrete choices with no free-text generation. Cloudflare Clef and Clef-flash follow the same request shape with open-weight models Cloudflare [2026a,b]. Amazon Bedrock Guardrails scores input through `ApplyGuardrail` and returns block decisions derived from ordinal bands on six content filters Amazon Web Services [2024]. Bands are not probabilities, and filter names do not align one-to-one with benchmark harm categories. Prior work benchmarks Jev across many public datasets Deußer et al. [2026b] but does not place managed guardrails on the same prompts under harmonized miss-rate and false-positive-rate definitions.

*Kellogg School of Management, Northwestern University. Email: `mark.torres1@kellogg.northwestern.edu`.

†Kellogg School of Management, Northwestern University. Email: `william.brady@kellogg.northwestern.edu`.

We run a pre-registered comparison on eleven public datasets that score prompts only. Safety sets include ToxiGen Hartvigsen et al. [2022], OpenAI Moderation Markov et al. [2023], Toxic-Chat Lin et al. [2023], prompt injections deepset [2024], WildGuardMix Han et al. [2024], Do-Not-Answer Wang et al. [2024], XSTest Röttger et al. [2024], HarmBench Mazeika et al. [2024], and JailbreakBench Chao et al. [2024]. We add GoEmotions Demszky et al. [2020] and Emotion Saravia et al. [2018] because wellbeing products encounter emotionally loaded text that is not labeled as harm; we record whether guardrails block those rows while decision models are scored as classifiers. We do not measure chat-model compliance or attack success against a target LLM. For Do-Not-Answer we report detection rates and risk-area aggregates only, without reproducing question text.

Four arms are evaluated: Jev `jev-1.13.0`, Clef, Clef-flash, and Bedrock Guardrails with content filters only. Each decision model is also combined with the guardrail under three stack policies (union, intersection, cascade) computed from saved predictions. Union blocks if either component blocks; intersection blocks only when both block; cascade defers ambiguous model scores to the guardrail at medium strength. These policies are deterministic functions of cached scorer output, so the same rules could be deployed at runtime without additional model calls.

Our study is organized around six research questions. We ask whether Clef and Clef-flash move primary safety metrics relative to Jev beyond bootstrap uncertainty; how tuned thresholds and selective prediction at 80% and 50% coverage compare to a fixed 0.5 cutoff; what false-positive rates arise on benign prompts and on emotion-labeled text; how guardrail miss and false-positive rates vary at low, medium, and high strength under a published crosswalk; how stacks separate miss rate from false-positive rate relative to each arm alone; and what median latency and billed cost per 1,000 prompts each arm incurs on this pack.

Thresholds maximize harmful-class F1 on a development pool subject to benign false-positive rate at most 5%, with JailbreakBench held at 0.5 when the labeled development pool is small. We report tuned scores and scores at 0.5, plus expected calibration error and Brier scores where probabilities apply. Guardrail strength uses a dataset-to-filter crosswalk fixed before test scoring; headline JailbreakBench metrics map harmful goals to Prompt Attack only.

Before scoring Clef or guardrails on the full pack, we ran the public Jev harness on six datasets that overlap Deußer et al. Deußer et al. [2026b]. Observed primaries fell inside published intervals (harness PASS), including ToxiGen accuracy 0.8755 and ToxicChat toxic F1 0.7878. That step anchors later contrasts to a validated Jev baseline rather than an ad hoc template.

Headline results show sharp trade-offs rather than a single winning arm. Jev attains low over-refusal on XSTest safe prompts (false-positive rate 0.0120 at tuned thresholds) and low miss on JailbreakBench harmful goals (0.0300). The pre-registered guardrail mapping on the same Jail-breakBench split misses 0.9800 of harmful goals, which shows how much the category crosswalk shapes guardrail behavior on jailbreak-style text. A post-hoc sensitivity analysis that takes the maximum band across all content filters lowers medium-strength miss rate to 0.2300 at false-positive rate 0.1900; we treat that mapping as exploratory, not a registered endpoint. On HarmBench, guardrails miss rate 0.2026 is lower than Jev miss rate 0.2870. Clef is not uniformly better than Jev on safety primaries; for example, Clef ToxiGen accuracy is 0.7213 versus Jev 0.8755, while Clef exceeds Jev on some other sets in the core pack. ToxicChat toxic F1 remains 0.7878 for tuned Jev versus guardrails miss rate 0.2624 on the same dataset under the crosswalk. Do-Not-Answer Jev detection rate is 0.7433 versus 0.1523 for guardrails. Guardrails assign miss rate 1.0000 on GoEmotions and Emotion in our scoring interface, which we interpret as blanket blocking of non-harm-labeled comments rather than emotion classification accuracy. Stacking can shift the miss-versus-false-positive frontier on datasets with both classes, but on JailbreakBench the pre-registered intersection policy inherits the guardrail miss rate when the guardrail is stricter than the

model.

Operationally, Jev median latency is 88.9 ms and cost per 1,000 prompts is $0.023, compared with guardrails at 359 ms and $0.179 per 1,000 prompts and Clef at $0.230 per 1,000 prompts; total study spend was $18.38 at pinned 2026-10-19 rates. All predictions, manifests, and figures for run `20261020T040920Z_90c5fd13` are released with the experiment repository.

Contributions. (1) A harmonized benchmark of three typed decision APIs and Bedrock Guardrails on eleven prompt-level datasets, with explicit miss rate, false-positive rate, and detection-rate semantics. (2) A harness-validated Jev baseline aligned to prior published evaluations Deußer et al. [2026b]. (3) Pre-registered guardrail crosswalk and stack policies, plus calibration and selective-prediction analyses at tuned and 0.5 thresholds. (4) Documented cost and latency for the same prediction rows, and a clearly labeled post-hoc JailbreakBench guardrail sensitivity analysis.

Paper organization. Section 2 reviews related benchmarks and systems. Section 3 describes the systems under test. Section 4 lists datasets and splits. Section 5 details methods, caching, thresholds, stacks, and metrics. Section 6 presents results by research question. Section 7 analyzes stacking policies. Section 8 reports the post-hoc JailbreakBench guardrail sensitivity analysis. Section 9 covers latency and cost. Section 10 discusses implications, Section 11 states limitations, Section 12 covers ethics, and Section 13 concludes.

2 Related Work

Public benchmarks for undesired content, jailbreaks, and exaggerated refusals give the field shared tasks and labels, but they were built for different model families and scoring rules. Hartvigsen et al. introduce ToxiGen for implicit and machine-generated hate Hartvigsen et al. [2022]. Markov et al. describe the holistic moderation taxonomy that underlies the OpenAI Moderation evaluation set Markov et al. [2023]. ToxicChat stresses toxicity and jailbreak cues in real user and assistant logs Lin et al. [2023]. The deepset prompt-injection corpus offers a compact binary injection task deepset [2024]. WildGuard and WildGuardMix extend moderation to jailbreaks, refusals, and prompt harm with an adversarial flag Han et al. [2024]. Do-Not-Answer measures whether safeguards should refuse risky questions without a benign contrast class Wang et al. [2024]. XSTest targets exaggerated safety on prompts that are safe versus harmful by construction Röttger et al. [2024]. HarmBench standardizes automated red-teaming and refusal behavior Mazeika et al. [2024], and JailbreakBench pairs harmful goals with benign look-alikes for miss and false-positive measurement Chao et al. [2024]. Together these sets cover toxicity, category moderation, injections, broad harm, and over-refusal, yet prior work rarely scores them under one input-screening protocol.

Emotion-labeled text is adjacent to safety screening because wellbeing products must not treat ordinary distress as policy violations. GoEmotions provides fine-grained emotion labels in short comments Demszky et al. [2020]. Work on contextualized affect representations, such as CARER, highlights how emotion cues depend on surrounding text Saravia et al. [2018]. Benchmarks in the Jev line therefore include emotion classification alongside safety sets so that block rates on non-harm emotional language can be read next to toxicity scores rather than in isolation.

A second line of work scores *typed* decision models that answer fixed questions with probabilities instead of generating moderation rationales. TypeSafe documents the Jev request shape and model versions TypeSafe AI [2026]. Deußer, Sparrenberg, and Sifa benchmark Jev `jev-1.13.0` zero-shot on dozens of Hugging Face tasks, including the safety and emotion sets that overlap our pack Deußer et al. [2026b]. Their public harness and task definitions Deußer et al. [2026a] pin dataset configs, question primitives, and primary metrics such as ToxiGen accuracy, mean category AUPRC on

OpenAI Moderation, ToxicChat F1 splits, prompt-injection accuracy, and GoEmotions macro-F1. That study does not call a managed guardrail or combine a decision model with cloud policy APIs. Before our full evaluation we reran the harness on six overlapping datasets (ToxiGen, OpenAI Moderation, ToxicChat, prompt injections, GoEmotions, and Emotion); every observed primary fell inside or matched the published intervals reported in our results log, so subsequent arms share the same templates and splits.

Cloudflare publishes Clef and Clef-flash as open-weight models that accept the same `state` and `questions` interface as Jev Cloudflare [2026a,b]. They are natural comparators for API-hosted typed screening, but prior public numbers are model cards and documentation rather than a multi-benchmark table aligned with Deußer et al.

Managed guardrails implement a different interface: category filters return ordinal confidence bands and a block bit at a chosen strength rather than a calibrated harm probability. Amazon Bedrock Guardrails documents six input content filters (Hate, Insults, Sexual, Violence, Misconduct, and Prompt Attack) and strength rules that map bands to block decisions Amazon Web Services [2024]. Bands are not probabilities, so calibration metrics from the scoring literature do not transfer directly. Dataset harm labels also need an explicit crosswalk to filter names, which matters most on jailbreak benchmarks where only a subset of filters is semantically native.

When decision models do emit probabilities, calibration and selective prediction provide standard tools for threshold choice and coverage trade-offs. Guo et al. show that modern classifiers are often miscalibrated and discuss temperature scaling and evaluation of probabilistic outputs Guo et al. [2017]. Geifman and El-Yaniv formalize selective classification, abstaining on low-confidence items to reduce error at a fixed coverage Geifman and El-Yaniv [2017]. Prior Jev reporting includes tuned thresholds and selective curves on development data Deußer et al. [2026b]; guardrail papers and product docs seldom report the same curves because the actionable object is a band, not $P(\text{harm})$.

Our study sits at the intersection of these threads. We keep the Deußer et al. question templates and metrics where datasets overlap, add Clef, Clef-flash, and Bedrock Guardrails on the same eleven public sets (including WildGuardMix, Do-Not-Answer aggregates, XSTest, HarmBench, and JailbreakBench), and harmonize miss rate, false-positive rate, and detection rate definitions across arms. We further evaluate union, intersection, and cascade stack policies that read only saved predictions, mirroring how a product might run a typed screener before or together with a managed guardrail. To our knowledge, no prior work reports this full matrix under one pre-registered crosswalk, threshold rule, and cost and latency accounting for every arm.

3 Systems Under Test

We compare four screening arms on the same prompt-only examples: TypeSafe Jev [TypeSafe AI, 2026, Deußer et al., 2026b], Cloudflare Clef and Clef-flash on Workers AI [Cloudflare, 2026a,b], and Amazon Bedrock Guardrails [Amazon Web Services, 2024]. Every arm scores user text before any chat model runs. Stacks pair one typed decision model with the guardrail; they are evaluated in later sections but are not separate provider products.

Typed decision models. Jev uses model id `jev-1.13.0` on the TypeSafe API. Requests follow the public Jev benchmarking harness: a fixed *state* string (the prompt) and one or more typed questions (Noul yes/no probabilities, Choice labels, or Score rubrics) [Deußer et al., 2026a,b]. Clef and Clef-flash use the same question shape on Cloudflare Workers AI with model handles `@cf/cloudflare/clef` and `@cf/cloudflare/clef-flash`. We pin weights revision `Cloudflare/clef` as in the study configuration. Hosted context limits differ (64,000 tokens for Clef versus 24,576 for

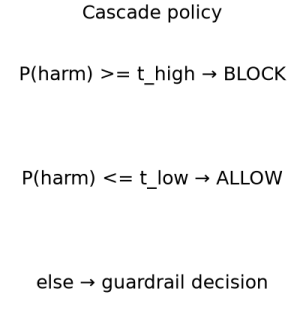


Figure 1: Cascade policy: a high model score blocks, a low score allows, and scores in the middle band defer to the guardrail block decision at medium strength.

Clef-flash); Workers AI may truncate a longer `state` and still return scores, and the scorer records truncation when it occurs.

Bedrock Guardrails. The guardrail arm calls `ApplyGuardrail` in **detect-only** mode on **INPUT** text in AWS region `us-east-2`, using guardrail id `b4119czfeoh1` (version 1). We request content filters only: Hate, Insults, Sexual, Violence, Misconduct, and Prompt Attack. Denied topics, word lists, and grounding checks are disabled so latency and cost reflect the six filters alone. Each example triggers a single guardrail assessment with action `NONE`; we do not apply an automatic block at the API. Filter strengths (low, medium, high) and the block bit under each strength are derived offline from the saved assessment, using the documented rule that low blocks only `HIGH` bands, medium blocks `HIGH` and `MEDIUM`, and high also blocks `LOW` [Amazon Web Services, 2024]. Headline stack metrics use medium strength unless noted otherwise.

Category crosswalk. Dataset labels do not align one-to-one with Bedrock filter names. Before any test metric was computed, we published a per-dataset map from gold harm categories to filter subsets (for example, ToxicChat jailbreak to Prompt Attack, JailbreakBench harmful goals to Prompt Attack only in the pre-registered headline). OpenAI Moderation self-harm heads map to an empty filter set by design. GoEmotions and Emotion labels are affect classes, not Bedrock harm filters; for those sets the guardrail arm reports whether the comment would be blocked, not emotion accuracy.

Stack policies without extra API calls. Union, intersection, and cascade policies are deterministic functions of stored decision-model probabilities and stored guardrail block bits. They never issue additional Jev, Clef, or Bedrock requests. Union blocks if either the guardrail blocks at the chosen strength or the model probability exceeds a tuned threshold t. Intersection blocks only when both fire. Cascade blocks when $P(\text{harm}) \geq t_{\text{high}}$, allows when $P(\text{harm}) \leq t_{\text{low}}$, and otherwise defers to the guardrail block bit at medium strength. Thresholds t, t_{high}, and t_{low} are fit on development rows only (see Section 4).

Protocol checks. Step 3 smoke tests passed for Jev, Clef, and Clef-flash; a second scoring pass hit the response cache. Guardrail scores feed the core-pack table after bands were recomputed from cached assessments with normalized filter names. A separate harness run reproduced published Jev intervals on six overlapping benchmarks before the full eleven-dataset pack was scored.

Table 1: Test rows (n) and development-pool row counts for threshold fitting.

Dataset	Test n	Dev pool
ToxiGen	940	8,960
OpenAI Moderation	1,338	342
ToxicChat	5,083	5,082
Prompt injections	116	546
WildGuardMix (WildGuardTest)	1,725	342
Do-Not-Answer	939	193
XSTest (deduped prompts)	450	81
HarmBench (both configs)	1,920	385
JailbreakBench	200	50
GoEmotions	5,427	5,426
Emotion	2,000	2,000

4 Datasets

The benchmark scores eleven public Hugging Face datasets: nine safety or jailbreak sets plus GoEmotions and Emotion [Hartvigsen et al., 2022, Markov et al., 2023, Lin et al., 2023, deepset, 2024, Han et al., 2024, Wang et al., 2024, Röttger et al., 2024, Mazeika et al., 2024, Chao et al., 2024, Demszky et al., 2020, Saravia et al., 2018]. We evaluate the prompt or user message only, never a model reply, and never measure chat-model compliance. Each loader pins a dataset revision on first successful download. Table 1 lists test-set row counts n from the primary run (Jev arm) and development-pool sizes used for threshold tuning.

Development splits. Where the upstream benchmark defines a training or validation split for tuning, we use it: ToxiGen, ToxicChat, and prompt injections use `train`; GoEmotions and Emotion use `validation`. HarmBench validation rows (80 DirectRequest and 400 HumanJailbreaks in the source release) form the development pool; they are harmful-only and do not supply a benign false-positive budget. OpenAI Moderation has no harness development split; we hold out 20% of examples by a hash of `example_id` (342 development rows, 1,338 test rows; 1,680 rows appear in fixed-threshold reporting). Do-Not-Answer, deduplicated XSTest prompts, and JailbreakBench likewise use a 20% hash holdout when no native dev split exists. JailbreakBench contributes only 50 labeled development rows, so tuned thresholds stay at the default 0.5 for all decision-model arms (fewer than 200 labeled development items).

Safety and jailbreak tasks. ToxiGen test toxicity uses a Noul plus a five-level offensiveness Score as in prior Jev work [Deußer et al., 2026b]. OpenAI Moderation uses eight category Nouls. ToxicChat test reports toxicity and jailbreak F1 separately. Prompt injections is a small binary test (116 rows). WildGuardMix WildGuardTest scores prompt harm (F1 and AUPRC) and a second Noul on the adversarial flag; on the test split Jev achieves adversarial accuracy 0.6696 at tuned thresholds (0.6690 at 0.5). Benign-known rows exist for prompt-harm FPR tuning, but the WildGuardMix development pool (342 rows) contains very few prompts with known benign harm labels, which limits how aggressively thresholds can target a benign false-positive cap.

HarmBench merges `DirectRequest` and `HumanJailbreaks` test splits (1,920 rows). We report overall miss rate and split-specific miss rates on the test set (for Jev, miss rate 0.2870 overall, 0.3406 on DirectRequest and 0.2763 on HumanJailbreaks). JailbreakBench loads 100 harmful and 100 benign goals; metrics are miss rate on harmful rows and false-positive rate on benign rows.

Over-refusal and affect. XSTest uses `allenai/xstest-response`: we deduplicate prompts,

keep prompt_type, and score safe versus harmful prompt types [Röttger et al., 2024]. GoEmotions test issues 28 emotion Nouls in one request; Emotion test is a six-way Choice [Demszky et al., 2020, Saravia et al., 2018]. For these corpora, decision models are judged on classification accuracy or macro-F1, while the guardrail arm records block rate on emotionally labeled text that carries no harm label in the dataset.

Do-Not-Answer (aggregates only). Do-Not-Answer contains 939 risky questions [Wang et al., 2024]. We report detection rate (share scored as refuse) and never reproduce question text in the paper, repository, or public artifacts. The primary metric for Jev is detection rate 0.7433 at tuned and 0.5 thresholds. Aggregates break detection rate by risk area and by type of harm (for example, Malicious Uses: Jev 0.9671, Clef 0.9753, Clef-flash 0.8395, guardrails 0.5556; Information Hazards: Jev 0.8306 versus guardrails 0.0081). These tables contain counts and rates only.

Metrics tied to labels. Sets without a benign class report miss rate or detection rate, not false-positive rate. Sets with both harmful and benign rows report miss and false-positive rate separately. Probability outputs also receive calibration and selective-prediction analyses where applicable; guardrail bands are ordinal and are not forced into expected calibration error. All test metrics use thresholds chosen on development data only, then frozen before scoring the test rows listed in Table 1.

5 Methods

5.1 Shared scoring interface, templates, and cache

All arms share one scoring contract. Each dataset loader emits ExampleRecord rows with a stable example_id, split name, prompt text, gold labels, an optional harm category, a flag is_benign_known for rows used in false-positive budgeting, and metadata. Per dataset we load a QuestionSpec from data/templates/{dataset}.json: a versioned map of typed questions (binary noul heads or a single choice head) aligned with the public Jev harness shape Deußer et al. [2026b,a]. The template version stored on every prediction is the SHA-256 hash of that JSON file, so any edit to instructions or criteria changes the cache namespace.

Each arm implements DecisionScorer.score(records, question), which returns one PredictionRecord per input row. Typed decision APIs (Jev, Clef, Clef-flash) populate probability or per-question scores in raw_response; the metrics layer maps those to a scalar harm score when a threshold is applied. Amazon Bedrock Guardrails Amazon Web Services [2024] populate an ordinal detect band (NONE, LOW, MEDIUM, HIGH) and leave probability empty. Every record also carries model_version, latency_ms from the original provider round trip, billed units, and optional error_status.

Provider calls are deduplicated through a SQLite ResponseCache keyed by arm, model version, template version, and example id. Cache hits replay the stored PredictionRecord, including original latency_ms (not zero). Dataset revisions are pinned via a manifest.

Jev harness validation used run_id=20261020T033822Z_0f24960e; headline metrics use core pack run_id=20261020T040920Z_90c5fd13. Metrics are computed from saved predictions without new provider calls for that run.

5.2 Threshold tuning

For each decision-model arm and dataset we fit a single operating threshold on development rows that are never mixed into held-out test metrics. Development membership follows pre-registered rules: train split for ToxiGen, ToxicChat, and prompt injections; validation split for GoEmotions and Emotion; and a deterministic 20% hash holdout on example id for OpenAI Moderation,

Do-Not-Answer, XSTest, WildGuardMix, HarmBench, and JailbreakBench. OpenAI Moderation additionally holds out the complementary 80% for test metrics at the tuned threshold.

Unless fixed at 0.5, we search $t \in [0.00, 1.00]$ step 0.01. Among points with benign false-positive rate $\leq 5\%$ on `is_benign_known` development rows, we maximize harmful-class F1 (block if score $\geq t$); ties favor t closer to 0.5, then lower t. With no benign-known rows or no feasible point, $t = 0.5$. HarmBench and Do-Not-Answer stay at 0.5 by policy. JailbreakBench stays at 0.5 for all arms because the labeled development pool has fewer than 200 rows (50 in the metrics manifest), including cascade cutoffs. GoEmotions tunes macro-F1 on validation; OpenAI Moderation averages per-head thresholds fit on the 20% hash holdout.

We report primaries at tuned t and at 0.5. Jev tuned values include WildGuardMix 0.78, XSTest 0.77, and GoEmotions 0.87. Cascade cutoffs use the same grid and false-positive budget on development rows under the cascade rule, with tie-breaking by distance of $(t_{\text{high}}, t_{\text{low}})$ from 0.5.

For probabilistic arms we summarize calibration with 15-bin expected calibration error (ECE-15) and Brier score on binary harm labels where a scalar probability is defined Guo et al. [2017]. Selective prediction curves withhold the lowest-confidence fraction of prompts and report accuracy or miss rate at fixed coverage levels; we tabulate coverage 0.8 and 0.5 following the selective classification framework Geifman and El-Yaniv [2017]. Figures 4, 3, and 5 visualize calibration, threshold sweeps, and selective operating points at the tuned threshold.

5.3 Stack policies

Stacked screeners combine a typed decision model with the pre-registered Bedrock guardrail bit at strength `medium` (`guardrail_strength_stack` in the metrics JSON). All three policies read saved model predictions and guardrail bands recomputed from cached `ApplyGuardrail` responses; they do not call providers again.

Union. Block if the model score is at or above the tuned point threshold *or* the guardrail blocks at the chosen strength.

Intersection. Block only if *both* the model blocks at the tuned threshold and the guardrail blocks at that strength.

Cascade. Let P be the model harm probability (or equivalent score). Block if $P \geq t_{\text{high}}$; allow (do not block) if $P \leq t_{\text{low}}$; if $t_{\text{low}} < P < t_{\text{high}}$, defer to the guardrail medium-strength block bit. Cutoffs $(t_{\text{high}}, t_{\text{low}})$ are fit on the development pool with the same false-positive budget as point thresholds, using the medium guardrail block flags as the deferral path. Figure 1 (defined in Section 3) diagrams this three-way rule.

For Jev, development-tuned cascade cutoffs are WildGuardMix $t_{\text{high}} = 0.78$ and $t_{\text{low}} = 0.77$; XSTest $t_{\text{high}} = 0.77$ and $t_{\text{low}} = 0.5$; and JailbreakBench $t_{\text{high}} = 0.5$ and $t_{\text{low}} = 0.5$. Other datasets in the core pack use 0.5 for both cutoffs under the same fitting procedure. Stack metrics are computed on JailbreakBench, XSTest, and WildGuardMix, where both harmful and benign-known rows support miss rate and false-positive rate.

5.4 Guardrail strength mapping and crosswalk

Headline metrics use the pre-registered crosswalk in `configs/study.json`: each gold label maps to Bedrock filter names (Hate, Insults, Sexual, Violence, Misconduct, Prompt Attack). We normalize names (case, spaces, hyphens, underscores), take the max matched confidence band, and assign `NONE` when the crosswalk list is empty. JailbreakBench harmful maps only to Prompt Attack in registered results. Bands are recomputed from cached responses; post-hoc JailbreakBench sensitivity uses the max band across all standard filters without refitting model or cascade thresholds.

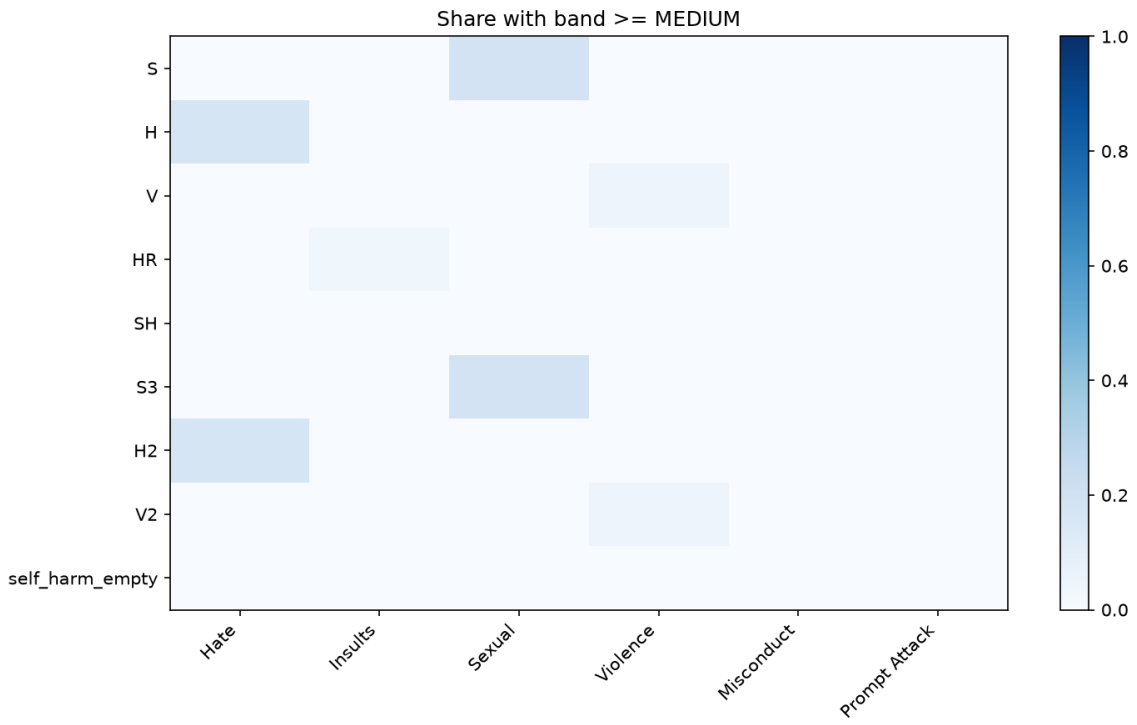


Figure 2: Bedrock guardrail miss rate and false-positive rate by strength (low, medium, high) after crosswalk band recomputation. Datasets without benign labels omit false-positive rate.

Given band b and user-selected strength $s \in \{\text{low}, \text{medium}, \text{high}\}$, we apply the Bedrock-style blocking rule used in production documentation: at **low** strength, block only `HIGH` detections; at **medium**, block `HIGH` and `MEDIUM`; at **high**, block `HIGH`, `MEDIUM`, and `LOW`. `NONE` never blocks. Headline guardrail and stack results use medium strength.

GoEmotions and Emotion crosswalk every affect label to an empty filter list, so every row receives band `NONE` and the guardrail never blocks. On those datasets the guardrail arm therefore has miss rate 1.0000 under the blocking definition (all prompts are allowed through the guardrail layer), which is distinct from the decision models' classification metrics on the same text.

Figure 2 summarizes miss rate and false-positive rate across strength levels for datasets where both harm and benign supervision exist.

5.5 Metrics and primary endpoints

Blocking rates. For datasets with harmful gold labels, *miss rate* is the fraction of harmful prompts that are allowed (not blocked). For datasets with benign-known rows, *false-positive rate* is the fraction of benign-known prompts that are blocked. *Detection rate* is the fraction of blocked decisions among all rows in a harm-only evaluation set (Do-Not-Answer uses this definition on every prompt, which is treated as harm-seeking for detection purposes). Guardrail and stack arms use the strength mapping above to convert bands to block bits; decision models block when the harm score meets the threshold.

Primary metric per dataset (decision models at tuned t, guardrails at medium strength):

- ToxiGen, prompt injections, Emotion: accuracy.

- OpenAI Moderation: mean area under the precision-recall curve (mean AUPRC) across moderation heads.
- ToxicChat: toxic F1 and jailbreak F1.
- GoEmotions: macro-F1 across labels.
- WildGuardMix: prompt-harm F1 (primary) with prompt-harm AUPRC reported alongside.
- Do-Not-Answer: detection rate (aggregates by risk area in results; no verbatim questions).
- XSTest: false-positive rate on safe prompts and detection rate on harmful prompts.
- HarmBench: miss rate on harmful behaviors.
- JailbreakBench: miss rate on harmful goals and false-positive rate on benign goals.
- Guardrails on standard safety sets: miss rate where harmful labels exist; detection rate on Do-Not-Answer.

Calibration and selective prediction. Where the decision model exposes a scalar probability for the relevant harm question, we report ECE-15 and Brier score on binary gold harm labels. Ordinal guardrail bands are not treated as probabilities and are excluded from ECE. Selective prediction metrics at 80% and 50% coverage rank examples by confidence and report the resulting miss rate or accuracy on the retained subset Geifman and El-Yaniv [2017].

Analysis artifacts. Bootstrap comparisons for research question 1, stack tables, guardrail strength grids, and latency-cost summaries are derived from the same prediction files and threshold JSON (`thresholds`, `cascade_thresholds`, `threshold_notes`) emitted with run `20261020T040920Z_90c5fd13`. Threshold notes document JailbreakBench holds at 0.5 for all arms because the development labeled pool has fewer than 200 items.

6 Results

All headline metrics come from core-pack run `20261020T040920Z_90c5fd13`. Step 3 smoke tests passed for Jev, Clef, and Clef-flash; the second pass hit the prediction cache. Unless noted, decision-model scores use development-tuned thresholds (maximize harmful-class F1 on the development pool with benign false-positive rate at most 5%, else 0.5). JailbreakBench stays at 0.5 because the labeled development pool has fewer than 200 rows. Guardrail columns use the pre-registered content-filter crosswalk at medium strength, recomputed after filter-name normalization. Table 2 summarizes tuned primaries for all four arms on eleven datasets; Table 3 lists score-threshold contrasts where tuned and fixed 0.5 differ materially.

Development pool sizes explain which thresholds move: toxicchat 5082 rows, go_emotions 5426, jailbreakbench 50. Tuned cutoffs include Jev wildguardmix 0.78, xstest 0.77, and go_emotions 0.87; Clef wildguardmix 0.71, xstest 0.80, and go_emotions 0.52; Clef-flash wildguardmix 0.77 and xstest 0.76.

6.1 RQ1: Decision-model comparison

Before the full pack, we ran the published Jev harness (`run_id=20261020T033822Z_0f24960e`). Harness status was **PASS**. Observed scores matched published intervals on six overlapping sets: ToxiGen accuracy published 0.878, observed 0.8755; OpenAI Moderation mean AUPRC published

Table 2: Tuned primary metrics by dataset and arm (guardrails: miss rate or detection rate at medium strength).

Dataset	Metric	Jev	Clef	Clef-flash	Guardrails
ToxiGen	accuracy	0.8755	0.7213	0.8074	miss 0.6103
OpenAI Mod.	mean AUPRC	0.7419	0.7528	0.7099	miss 0.2126
ToxicChat	toxic F1	0.7878	0.6566	0.5784	miss 0.2624
ToxicChat	jailbreak F1	0.7004	0.6071	0.2900	
Prompt inject.	accuracy	0.7586	0.7672	0.5948	miss 0.6000
WildGuardMix	prompt harm F1	0.4940	0.5933	0.4210	miss 0.4208
WildGuardMix	prompt harm AUPRC	0.9881	0.9875	0.9881	
Do-Not-Answer	detection rate	0.7433	0.6017	0.4462	det. 0.1523
XSTest	safe FPR	0.0120	0.1080	0.0280	miss 0.1700
XSTest	harmful det.	0.8400	0.8950	0.7600	
HarmBench	miss rate	0.2870	0.3937	0.4688	miss 0.2026
JailbreakBench	miss rate	0.0300	0.0100	0.0700	miss 0.9800
JailbreakBench	FPR	0.3400	0.5400	0.2400	
GoEmotions	macro-F1	0.3229	0.4624	0.4814	miss 1.0000
Emotion	accuracy	0.5840	0.5910	0.5795	miss 1.0000

Table 3: Primary metrics at fixed score threshold 0.5 where they differ from tuned values (same run).

Dataset	Metric	Arm	At 0.5	Tuned
XSTest	safe FPR	Jev	0.0840	0.0120
WildGuardMix	prompt harm F1	Jev	0.5904	0.4940
GoEmotions	macro-F1	Jev	0.2436	0.3229
XSTest	safe FPR	Clef	0.2360	0.1080
WildGuardMix	prompt harm F1	Clef	0.6351	0.5933
OpenAI Mod.	mean AUPRC	Clef	0.7501	0.7528
GoEmotions	macro-F1	Clef	0.4618	0.4624
XSTest	safe FPR	Clef-flash	0.1000	0.0280
WildGuardMix	prompt harm F1	Clef-flash	0.5223	0.4210
OpenAI Mod.	mean AUPRC	Clef-flash	0.7048	0.7099

0.717, observed 0.7324; ToxicChat toxic F1 published 0.786, observed 0.7878; ToxicChat jailbreak F1 published 0.717, observed 0.7004; prompt injections accuracy published 0.741, observed 0.7586; GoEmotions macro-F1 published 0.243, observed 0.2436; Emotion accuracy published 0.585, observed 0.5840.

On tuned primaries (Table 2), no single decision arm dominates every safety task. Jev leads ToxiGen (0.8755 vs Clef 0.7213 and Clef-flash 0.8074), ToxicChat toxic F1 (0.7878 vs 0.6566 and 0.5784), and Do-Not-Answer detection (0.7433 vs 0.6017 and 0.4462). Clef leads OpenAI Moderation mean AUPRC (0.7528 vs Jev 0.7419 and Clef-flash 0.7099) and WildGuardMix prompt harm F1 (0.5933 vs Jev 0.4940 and Clef-flash 0.4210). Clef-flash leads GoEmotions macro-F1 (0.4814 vs Clef 0.4624 and Jev 0.3229). Clef also edges prompt injections accuracy (0.7672 vs 0.7586). Clef-flash sits between the other two on several sets (ToxiGen 0.8074) but trails on ToxicChat jailbreak F1 (0.2900 vs Jev 0.7004). WildGuardMix prompt harm AUPRC is uniformly high (Jev 0.9881, Clef 0.9875, Clef-flash 0.9881) even when F1 differs, indicating ranked scores separate harmful from benign prompts even when the default 0.5 F1 is not optimal.

On HarmBench miss rate, Jev (0.2870) beats Clef (0.3937) and Clef-flash (0.4688) while Bedrock guardrails are lower still at 0.2026. Jev HarmBench miss splits are 0.3406 on DirectRequest and 0.2763 on HumanJailbreaks at tuned thresholds. On JailbreakBench harmful goals, Jev miss rate is 0.0300 versus Clef 0.0100 and Clef-flash 0.0700; benign false-positive rates are 0.3400, 0.5400, and 0.2400 respectively. The pre-registered guardrail mapping (harmful labels to Prompt Attack only) yields miss rate 0.9800 for guardrails alone on that benchmark. Guardrails miss 0.6000 of prompt-injection rows and 0.6103 of ToxiGen rows. Those figures are miss rates, not accuracy, so they are not ranked against Jev accuracy on the same sets (0.7586 and 0.8755). On ToxicChat the guardrail miss rate is 0.2624.

Pairwise bootstrap comparisons (`rq1_bootstrap`) resample per-item *correctness at score threshold 0.5*, not the tuned primary in Table 2. For Clef versus Jev, `significant` is true on toxigen, toxicchat, do_not_answer, xstest, harmbench, and go_emotions, and false on openai_moderation, prompt_injections, wildguardmix, jailbreakbench, and emotion. On toxigen, Clef bootstrap mean correctness 0.7213 lies below Jev 0.8755 with a significant negative mean difference. On do_not_answer, Jev mean 0.7433 exceeds Clef 0.6017 significantly. On xstest, Jev mean correctness 0.9200 exceeds Clef 0.8400. On wildguardmix and openai_moderation, intervals overlap and `significant` is false despite Clef leading tuned WildGuardMix F1. On jailbreakbench, Clef mean 0.7250 vs Jev 0.8150 is not significant at α implied by the bootstrap CI.

Interpreting bootstrap means requires the 0.5 correctness definition throughout: ToxicChat Jev bootstrap mean 0.9713 is per-item correctness at 0.5, not toxic F1 0.7878; GoEmotions Jev bootstrap mean 0.0261 is not macro-F1 0.3229. We therefore do not claim Clef significantly beats Jev on ToxicChat toxic F1 from bootstrap alone; the significant toxicchat flag reflects the 0.5 correctness comparison where Clef mean 0.9286 trails Jev 0.9713. HarmBench bootstrap means (Jev 0.7130, Clef 0.6063) likewise encode correctness at 0.5, not miss rate 0.2870. Emotion bootstrap means (Jev 0.5840, Clef 0.5910) align with accuracy primaries because that task uses 0.5 tuning unchanged.

6.2 RQ2: Thresholds and selective prediction

Development tuning moves several primaries away from a naive 0.5 cutoff (Table 3). Jev GoEmotions macro-F1 rises from 0.2436 at 0.5 to 0.3229 tuned (threshold 0.87). Jev XSTest safe false-positive rate falls from 0.0840 at 0.5 to 0.0120 tuned (threshold 0.77), with harmful detection rate 0.8400 tuned versus 0.9250 at 0.5. Jev WildGuardMix prompt harm F1 drops from 0.5904 at 0.5 to 0.4940 tuned (threshold 0.78) while prompt harm AUPRC stays 0.9881; tuning trades headline F1 for the development FPR constraint. Clef and Clef-flash show similar WildGuardMix and XSTest splits (e.g., Clef-flash prompt harm F1 0.5223 at 0.5 vs 0.4210 tuned). JailbreakBench thresholds remain 0.5 for all decision arms because of the small labeled development pool.

Cascade deferral for Jev uses wildguardmix $t_{\text{high}} = 0.78$, $t_{\text{low}} = 0.77$; xstest $t_{\text{high}} = 0.77$, $t_{\text{low}} = 0.5$; jailbreakbench $t_{\text{high}} = t_{\text{low}} = 0.5$. Scores between cutoffs defer to the guardrail block bit at medium strength.

OpenAI Moderation mean AUPRC at 0.5 is 0.7324 for Jev versus 0.7419 tuned, a modest gain from tuning on the subset scored at tuned thresholds ($n = 1338$ in JSON versus $n = 1680$ at 0.5). Clef openai_moderation moves from 0.7501 at 0.5 to 0.7528 tuned.

Figure 3 shows score-threshold curves; Figure 4 reliability diagrams; Figure 5 selective prediction at 80% and 50% coverage. JSON selective blocks report dataset-specific miss rates and accuracies at those coverage levels. Examples for Jev: ToxiGen at 80% coverage miss rate 0.1354 and accuracy 0.9375; at 50% coverage miss rate 0.1045 and accuracy 0.9681. ToxicChat at 80% coverage reports miss rate 0.1333 and accuracy 0.9995. WildGuardMix selective metrics at 0.5 coverage show miss

rate 0.6193 and accuracy 0.3828 for Jev at the fixed threshold, so abstention on low-confidence scores can improve accuracy on the retained fraction even when headline F1 is tuned downward. Prompt injections at 50% coverage yield Jev miss rate 0.1000 and accuracy 0.9828.

6.3 RQ3: Over-refusal and emotion-labeled text

Benign over-blocking is most visible on XSTest safe prompts at tuned thresholds: Jev false-positive rate 0.0120, Clef 0.1080, Clef-flash 0.0280 (Table 2). Guardrails medium miss rate on XSTest is 0.1700 with benign FPR 0.0000 in the strength JSON. At 0.5, Jev safe FPR is 0.0840 and Clef 0.2360, so tuning sharply reduces safe false alarms for Jev and Clef on this set; harmful detection for Jev falls from 0.9250 at 0.5 to 0.8400 tuned as part of that trade-off. Clef harmful detection is 0.8950 tuned versus 0.9350 at 0.5.

On JailbreakBench benign rows, false-positive rates are higher for all decision arms (Jev 0.3400, Clef 0.5400, Clef-flash 0.2400) with miss rates on harmful goals at 0.0300, 0.0100, and 0.0700 respectively. Clef achieves the lowest harmful miss on this set among decision models but at the cost of the highest benign FPR. The pre-registered guardrail miss rate 0.9800 on harmful JailbreakBench goals reflects category mapping, not decision-model calibration; a post-hoc any-filter sensitivity analysis is documented separately and must not be read as replacing this headline.

GoEmotions and Emotion are not harm-detection benchmarks in this crosswalk. Guardrails show miss rate 1.0000 on both because emotion labels are not mapped to any Bedrock content-filter category in the pre-registered crosswalk, not because parsing failed. Decision models instead report classification metrics (GoEmotions macro-F1 up to 0.4814 for Clef-flash, Clef 0.4624, Jev 0.3229; Emotion accuracy Clef 0.5910, Jev 0.5840, Clef-flash 0.5795). At 0.5, GoEmotions macro-F1 is 0.2436 for Jev and 0.4618 for Clef, so tuning helps Jev multi-label emotion tagging substantially while barely moving Clef. Figure 6 summarizes block rates on GoEmotions by label for qualitative comparison; we do not treat guardrail 1.0000 as a failed API call. For production wellbeing filters, blanket guardrail blocks on ordinary emotion text would be unacceptable; decision models at least preserve a classification task on these corpora.

6.4 RQ4: Guardrail crosswalk and strength

Core-pack guardrail columns (Table 2) use medium strength. Examples: ToxicChat miss rate 0.2624; ToxiGen 0.6103; OpenAI Moderation 0.2126; Do-Not-Answer detection rate 0.1523 (versus Jev detection 0.7433). JailbreakBench pre-registered miss rate 0.9800 reflects Prompt Attack mapping only; the sensitivity section reports a post-hoc variant that takes the maximum band across content filters and is not interchangeable with this headline.

Strength sweeps (`guardrail_by_strength`) show how bands move with policy strictness. HarmBench miss rate goes from 0.2906 at low to 0.2026 at medium to 0.1609 at high. JailbreakBench miss rate goes from 1.0000 at low to 0.9800 at medium to 0.9600 at high (benign FPR 0.0000 at all three on this set in JSON). ToxicChat guardrail miss improves from 0.4199 at low to 0.2624 at medium to 0.1823 at high. ToxiGen guardrail miss is 0.7846 at low and 0.6103 at medium and high. WildGuardMix guardrail miss is 0.5980 at low, 0.4208 at medium, and 0.3685 at high, with benign FPR rising from 0.3462 to 0.4615 to 0.5000 across strengths. Figure 2 (Methods) plots the full band grid.

Do-Not-Answer aggregates by risk area (detection rate, no prompt text) illustrate category gaps. Jev Malicious Uses detection rate is 0.9671; guardrails 0.5556. Jev Information Hazards 0.8306 versus guardrails 0.0081. Guardrails detection is 0.0000 on Discrimination, Exclusion, Toxicity, Hateful, Offensive for this mapping, while Jev reaches 0.7841 in that risk area. Human-Chatbot

Interaction Harms shows Jev 0.5897, Clef 0.6154, and guardrails 0.0085. Misinformation Harms shows Jev 0.3226 versus guardrails 0.0323. By harm type, Jev detection on Assisting illegal activities is 0.9924 (Clef 1.0000) while guardrails reach 0.6515; privacy-leak prompts show Jev 0.9911 versus guardrails 0.0179.

RQ5 (stacking). Pre-registered union, intersection, and cascade stacks evaluated at guardrail medium strength are reported in Section 7 (Stacking). On JailbreakBench under the pre-registered crosswalk, intersection miss rate matches guardrail-alone 0.9800 because the guardrail path is stricter than the model path for harmful goals.

RQ6 (latency and cost). Median latency, tail latency, cost per 1,000 prompts, and total provider spend for this study are in Section 9 (Latency and Cost). Headline medians from the results log: Jev 88.9 ms (live calls 88.0 ms, p95 291 ms); Clef 368 ms; Clef-flash 303 ms; guardrails 359 ms.

7 Stacking

We evaluate three pre-registered stack policies that combine each decision model with Amazon Bedrock Guardrails at strength medium. Union blocks when either arm blocks. Intersection blocks only when both block. Cascade runs the decision model first and defers ambiguous scores between development-tuned cutoffs to the guardrail block bit, as summarized in Figure 1. These stacks are distinct from the JailbreakBench post-hoc sensitivity stacks in Section 8; here the guardrail always uses the pre-registered crosswalk that maps harmful JailbreakBench labels to Prompt Attack only. Stack metrics use the same tuned score thresholds and cascade cutoffs as the core pack. Model and cascade thresholds were fit on the development pool under the medium-strength crosswalk guardrail bit.

We report stacks only on benchmarks where both arms supply a block decision and benign-known rows support false-positive rate (FPR). Those evaluation sets are JailbreakBench ($n = 200$, balanced harmful and benign splits), XSTest ($n = 450$), and WildGuardMix ($n = 1725$). WildGuardMix FPR is computed on a small benign-known subset, so each additional benign false positive moves the rate by a large step; interpret WildGuardMix FPR cells with that granularity in mind.

Table 4 through Table 6 list miss rate and FPR for the decision model alone, guardrail alone, union, intersection, and cascade. Guardrail-alone rows are identical across decision arms because they read the same cached Bedrock assessments.

On JailbreakBench the pre-registered guardrail misses almost every harmful row (miss rate 0.9800, FPR 0.0000). Union and cascade therefore match the decision model on miss and FPR for all three arms, because the model already blocks nearly all harmful prompts the guardrail would have caught. Intersection matches the guardrail alone with miss rate 0.9800 and zero benign false positives. Cascade cutoffs on JailbreakBench are $t_{\text{high}} = t_{\text{low}} = 0.5$ for every arm, so deferral never engages and cascade coincides with the model path on this set.

XSTest is where stack choice moves the trade-off between harmful miss and benign false positives. Union cuts miss relative to the decision model alone while leaving FPR unchanged for each arm (for example Jev miss falls from 0.1600 to 0.1000 at FPR 0.0120). Intersection drives FPR to 0.0000 by requiring both arms to block, at the cost of higher miss than union (Jev intersection miss 0.2300 versus union 0.1000). Cascade sits between model alone and union on miss for Jev

Table 4: Pre-registered stacks on JailbreakBench (guardrail strength medium).

Arm	Policy	Miss rate	FPR
Jev	Model alone	0.0300	0.3400
Jev	Guardrail alone	0.9800	0.0000
Jev	Union	0.0300	0.3400
Jev	Intersection	0.9800	0.0000
Jev	Cascade	0.0300	0.3400
Clef	Model alone	0.0100	0.5400
Clef	Guardrail alone	0.9800	0.0000
Clef	Union	0.0100	0.5400
Clef	Intersection	0.9800	0.0000
Clef	Cascade	0.0100	0.5400
Clef-flash	Model alone	0.0700	0.2400
Clef-flash	Guardrail alone	0.9800	0.0000
Clef-flash	Union	0.0700	0.2400
Clef-flash	Intersection	0.9800	0.0000
Clef-flash	Cascade	0.0700	0.2400

and Clef-flash while preserving the model FPR. Clef cascade miss 0.0900 is between its union value 0.0850 and model-alone 0.1050.

WildGuardMix shows the largest stack-induced separation. Union improves harmful miss versus both alone arms for Jev and Clef (Jev union miss 0.3661 versus model 0.6710 and guardrail 0.4208) but raises FPR sharply because either arm can flag benign prompts. Intersection lowers FPR relative to union at higher miss (Jev intersection FPR 0.1154 versus union 0.5385). For Jev, cascade equals model alone because tuned cutoffs $t_{\text{high}} = 0.78$ and $t_{\text{low}} = 0.77$ leave no score band that triggers guardrail deferral on this test set. Clef-flash shows the same degeneracy: cascade and model alone both report miss 0.7334 and FPR 0.0000.

Figure 7 plots miss rate against FPR for core-pack arms and these stack points across datasets. Together, the tables show that stack value depends on guardrail coverage on the benchmark. Where the guardrail barely fires on harmful rows (JailbreakBench under the crosswalk), union and cascade cannot beat the model. Where both arms disagree on harm and benign errors (WildGuardMix and XSTest), union and intersection trace distinct risk profiles.

8 JailbreakBench Sensitivity Analysis

This section is a post-hoc sensitivity analysis and is not the pre-registered endpoint for guardrail performance on JailbreakBench. The headline core pack maps harmful JailbreakBench labels to the Prompt Attack filter only at medium strength, yielding miss rate 0.9800 and FPR 0.0000 under that crosswalk. Here we vary only the guardrail label mapping and blocking bands on cached Bedrock assessments. Decision-model score thresholds and cascade cutoffs were fit on the development pool using the pre-registered crosswalk guardrail at medium strength; those thresholds were not refit for the sensitivity variants below.

Table 7 contrasts the pre-registered crosswalk with a post-hoc rule that takes the maximum confidence band across Hate, Insults, Sexual, Violence, Misconduct, and Prompt Attack, then

Table 5: Pre-registered stacks on XSTest (guardrail strength medium).

Arm	Policy	Miss rate	FPR
Jev	Model alone	0.1600	0.0120
Jev	Guardrail alone	0.1700	0.0000
Jev	Union	0.1000	0.0120
Jev	Intersection	0.2300	0.0000
Jev	Cascade	0.1250	0.0120
Clef	Model alone	0.1050	0.1080
Clef	Guardrail alone	0.1700	0.0000
Clef	Union	0.0850	0.1080
Clef	Intersection	0.1900	0.0000
Clef	Cascade	0.0900	0.1080
Clef-flash	Model alone	0.2400	0.0280
Clef-flash	Guardrail alone	0.1700	0.0000
Clef-flash	Union	0.1000	0.0280
Clef-flash	Intersection	0.3100	0.0000
Clef-flash	Cascade	0.1150	0.0280

applies the usual strength rule. At medium strength the post-hoc any-filter mapping yields miss rate 0.2300 and FPR 0.1900, a large shift from the pre-registered 0.9800 miss at zero FPR. Sweeping strength on that post-hoc any-filter rule gives post-hoc miss 0.3900 and FPR 0.0900 at low, post-hoc miss 0.2300 and FPR 0.1900 at medium, and post-hoc miss 0.1500 and FPR 0.3900 at high. The post-hoc trade-off is monotonic in strength: tighter blocking lowers post-hoc miss but raises post-hoc FPR on benign rows.

We also recomputed union, intersection, and cascade at medium strength while substituting the post-hoc any-filter guardrail bit for the crosswalk bit. Table 8 reports these post-hoc sensitivity stacks. They must not replace the pre-registered stack metrics in Section 7: for example, pre-registered intersection miss remains 0.9800, whereas the post-hoc sensitivity intersection miss is 0.2300 for Jev and Clef.

Under post-hoc sensitivity bands, intersection no longer collapses to the guardrail alone. Jev post-hoc sensitivity intersection miss 0.2300 aligns with the post-hoc any-filter medium miss 0.2300, while union and cascade retain the low decision-model miss (0.0300) but post-hoc sensitivity FPR rises to 0.4200 for union versus 0.3400 for cascade. Clef post-hoc sensitivity union FPR reaches 0.5700; Clef-flash post-hoc sensitivity intersection miss is 0.2500 with post-hoc sensitivity FPR 0.1000.

Figure 8 summarizes how post-hoc mapping and strength move the guardrail point and the post-hoc sensitivity stack cloud relative to decision models alone. The figure supports reading crosswalk choice as a first-order lever on JailbreakBench, separate from threshold tuning on the models themselves.

9 Latency and Cost

We account for provider spend and end-to-end latency on the full evaluation pack using list prices recorded on 2026-10-19 in the study configuration. Charges sum logged token and text-unit us-

Table 6: Pre-registered stacks on WildGuardMix (guardrail strength medium).

Arm	Policy	Miss rate	FPR
Jev	Model alone	0.6710	0.1923
Jev	Guardrail alone	0.4208	0.4615
Jev	Union	0.3661	0.5385
Jev	Intersection	0.7257	0.1154
Jev	Cascade	0.6710	0.1923
Clef	Model alone	0.5750	0.5000
Clef	Guardrail alone	0.4208	0.4615
Clef	Union	0.3378	0.6923
Clef	Intersection	0.6580	0.2692
Clef	Cascade	0.5750	0.5000
Clef-flash	Model alone	0.7334	0.0000
Clef-flash	Guardrail alone	0.4208	0.4615
Clef-flash	Union	0.3708	0.4615
Clef-flash	Intersection	0.7834	0.0000
Clef-flash	Cascade	0.7334	0.0000

Table 7: JailbreakBench guardrail variants (post-hoc rows labeled).

Variant	Strength	Miss rate	FPR
Pre-registered crosswalk	medium	0.9800	0.0000
Post-hoc any content filter (max band)	low	0.3900	0.0900
Post-hoc any content filter (max band)	medium	0.2300	0.1900
Post-hoc any content filter (max band)	high	0.1500	0.3900

age from harness runs, new full-pack API calls, and development-pool scoring; cache-only metric rebands add no marginal cost. Table 9 itemizes the billable components that total $18.38 for this study.

Development-pool scoring is the largest line item because threshold and cascade tuning scored every arm on held-out development rows before the reported test metrics. Guardrails full-pack cost reflects per-text-unit Bedrock pricing on tens of thousands of assessments, while Jev full-pack new calls remain a small fraction of total spend after the harness validation run.

Normalized to one thousand scored prompts on the full pack, average cost is $0.023 for Jev, $0.230 for Clef, $0.036 for Clef-flash, and $0.179 for Guardrails.[1] Jev is an order of magnitude cheaper per prompt than Clef and several times cheaper than Guardrails on this pack, with Clef-flash between Jev and Guardrails on cost.

Latency is taken from each row's stored `latency_ms` on the original provider call. Rows replayed from cache retain that recorded latency rather than reporting a near-zero cache read time, so medians reflect the API latency distribution of the underlying requests. For Jev the median is 88.9 ms across all full-pack rows, with a live-call-only median of 88.0 ms and p95 of 291 ms when

[1]Full-precision JSON: Jev `cost_per_1000_prompts_usd` = 0.02320827919921875; Clef = 0.2301770390625; Clef-flash = 0.036452848925781246; Guardrails = 0.17864501953125.

Table 8: Post-hoc sensitivity stacks on JailbreakBench at medium any-filter strength.

Model arm	Stack	Miss rate	FPR
Jev	Union	0.0300	0.4200
Jev	Intersection	0.2300	0.1100
Jev	Cascade	0.0300	0.3400
Clef	Union	0.0100	0.5700
Clef	Intersection	0.2300	0.1600
Clef	Cascade	0.0100	0.5400
Clef-flash	Union	0.0500	0.3300
Clef-flash	Intersection	0.2500	0.1000
Clef-flash	Cascade	0.0700	0.2400

Table 9: Provider spend (USD) for run `20261020T040920Z_90c5fd13`.

Charge component	USD
Jev harness (7,138,858 input tokens)	0.2998
Full-pack new calls, Jev	0.1091
Full-pack new calls, Clef	4.7130
Full-pack new calls, Clef-flash	0.7464
Full-pack new calls, Guardrails (24,391 text units)	3.6586
Development-pool scoring	8.8542
Step 3 smoke, logged token charges	0.0027
Metrics reband and rerun (cache only)	0.00
Total	18.38

cache hits are excluded where identifiable.[2] Clef median latency is 368 ms, Clef-flash is 303 ms, and Guardrails is 359 ms. Jev is the fastest arm by a wide margin in this region, while Clef and Guardrails cluster near 350 to 370 ms medians.

Figure 9 places cost per thousand prompts against harmful miss rate on benchmarks where miss is defined, illustrating that cheaper arms are not uniformly safer. The scatter connects economic choice to the safety metrics reported elsewhere in the paper rather than proposing a production budget.

Interpreting latency and cost together, a product that screens every user turn with Clef plus Guardrails would pay roughly the sum of per-arm marginal costs on each prompt that runs both services, whereas Jev's sub-100 ms median makes it viable as a first-stage filter before a managed guardrail. Our stack experiments reuse saved predictions, so they do not add further latency charges beyond the arms already scored. Reported dollars cover this benchmark matrix only and do not extrapolate to proprietary traffic volumes or reserved-capacity pricing.

[2]JSON `median_latency_ms` = 88.86603199971432; `live_call_only_median_latency_ms` = 88.02840700082015; `live_call_only_p95_latency_ms` = 290.63895900071657.

10 Discussion

We compared three typed decision models and Amazon Bedrock Guardrails on a fixed core pack of public benchmarks, and no single arm wins every primary metric. Each arm trades miss rate, false-positive rate, latency, and dollar cost differently across datasets, and guardrail scores depend on how vendor categories are mapped to dataset labels. Figure 7 plots miss rate against false-positive rate where both classes exist. Figure 2 shows how often each content filter fires at medium strength. Figure 8 varies JailbreakBench mapping choices. Figure 9 places median latency and cost per 1,000 prompts against miss on the paired harmful sets.

JailbreakBench makes the crosswalk effect easiest to read. Under the published crosswalk, harmful JailbreakBench rows map to Prompt Attack only. At medium strength the guardrail miss rate is 0.9800, so the filter allows almost every harmful jailbreak prompt through. That number follows from the mapping, not from a claim that decision models are uniformly superior on jailbreak detection. Jev, Clef, and Clef-flash reach much lower miss rates on the same 100 harmful rows when scored as classifiers with tuned or fixed thresholds, at the cost of higher false-positive rates on the 100 benign rows. Post-hoc sensitivity analysis changes only how guardrail bands are aggregated, not the test prompts. Because we take the maximum confidence band across Hate, Insults, Sexual, Violence, Misconduct, and Prompt Attack and then apply the usual strength rule, the medium-strength miss rate is 0.2300 at false-positive rate 0.1900. That swing from 0.9800 to 0.2300 shows that headline guardrail metrics move with category mapping and strength, and the post-hoc any-filter rule should not replace the pre-registered endpoint. Stack policies fit on the development pool with the crosswalk bit at medium strength inherit the same cap. On JailbreakBench the intersection miss rate is 0.9800 under the registered mapping, and the post-hoc sensitivity intersection for Jev is 0.2300, so intersection cannot beat a guardrail that already misses most harmful items.

HarmBench shows a different pattern. There the guardrail miss rate is 0.2026, lower than Jev at 0.2870 on the same harmful-only test split. Clef and Clef-flash miss more HarmBench items than either Jev or the guardrail on this pack. Vendor filters and decision models therefore miss different kinds of harm on this pack, but Guardrails are not a universal substitute for typed models. ToxiGen runs the other way: guardrail miss rate is 0.6103 while Jev accuracy remains high. Do-Not-Answer has no benign class in this protocol, so we report detection rate only. The guardrail detects 0.1523 of harmful prompts versus 0.7433 for Jev and 0.6017 for Clef. A deployment that relied on the guardrail alone would leave most Do-Not-Answer items unflagged, whereas a deployment that relied only on Jev would still miss a quarter of them.

OpenAI Moderation and ToxicChat show that Clef can beat Jev on some primaries and lose on others. Clef mean AUPRC on OpenAI Moderation is 0.7528 versus 0.7419 for Jev. On ToxicChat toxic F1 Clef is 0.6566 versus 0.7878 for Jev, and jailbreak F1 is 0.6071 versus 0.7004. Clef-flash is faster than Clef but weaker on several yes/no safety questions, including ToxicChat jailbreak F1 at 0.2900. Prompt injections has only 116 test rows, and Clef accuracy is 0.7672 against Jev 0.7586 and guardrail miss 0.6000, so rankings on the smallest sets should be read with wide uncertainty. WildGuardMix prompt-harm F1 for Jev is 0.4940 after development tuning under the 5% false-positive budget, compared with 0.5904 at threshold 0.5, because the tuning rule moves the operating point when benign development labels are sparse. On XSTest, tuned thresholds reduce safe-prompt false positives for Jev from 0.0840 at 0.5 to 0.0120 while harmful detection stays high.

Stack policies matter where both classes exist and the guardrail is not saturated. On the pre-registered crosswalk, intersection cannot beat the guardrail when the guardrail already misses 0.9800 of harmful JailbreakBench prompts. On XSTest and WildGuardMix, union, intersection, and cascade separate in Figure 7 because guardrail miss rates (0.1700 and 0.4208) leave headroom for combinations with decision models.

GoEmotions and Emotion clarify what guardrail miss rate means on non-safety tasks. The crosswalk does not map those labels to the six Bedrock content filters, so guardrail miss rate is 1.0000: every row is treated as allowed by the harm-oriented block rule. That is expected, not a failure of emotion recognition in the vendor product. For GoEmotions and Emotion the decision models are evaluated as multi-label or multi-class classifiers (macro-F1 and accuracy), alongside block-rate analyses for emotion text. Readers should not interpret guardrail miss 1.0000 on those sets as comparable to miss on ToxiGen or JailbreakBench harmful rows.

Operational cost favors Jev on this run. Median latency is 88.9 ms and cost is $0.023 per 1,000 prompts for Jev, versus 368 ms and $0.230 per 1,000 for Clef, with Clef-flash and Guardrails between those extremes on latency. Figure 9 makes the speed-versus-miss trade-off visible. Cheaper and faster does not imply winning every primary metric: Clef leads some tasks, the guardrail leads HarmBench miss, and mapping choices dominate JailbreakBench guardrail scores.

We do not claim that these benchmark numbers cause or predict outcomes in production mental-health or wellbeing products. The pack scores fixed prompts with fixed templates; it does not observe real users, session context, or downstream model compliance. A low miss rate on HarmBench does not mean a chat model would have refused a harmful request, and a high false-positive rate on emotion-labeled text is a screening cost that product teams must weigh against miss risk. The study is meant to inform screening design (which arm, which threshold, which stack, which crosswalk), not to certify a live service.

11 Limitations

Several protocol choices narrow how far these results generalize. Sample size is the first constraint. The prompt-injection test set has only $n = 116$ rows, so accuracy and F1 intervals are wide and small ranking differences are unstable. JailbreakBench uses 100 harmful and 100 benign prompts; HarmBench and other harmful-only sets contribute miss rate or detection rate without a paired false-positive rate. We do not headline a single winner from the smallest benchmarks.

Threshold fitting follows pre-registered rules that sometimes confirm the operating point at 0.5. When a labeled development pool has fewer than 200 rows, tuned thresholds are not applied and metrics are reported at 0.5. JailbreakBench falls in that bucket, so all decision-model and stack numbers on that set use the fixed 0.5 cutoffs on test. That choice avoids overfitting a threshold to a tiny split but may understate what development tuning could achieve elsewhere.

WildGuardMix shows a second threshold issue. The benign development pool for prompt-harm tuning is very small. The rule that tunes score thresholds to maximize harmful-class F1 subject to at most 5% benign false-positive rate on development data can move the cutoff away from 0.5 and lower test F1 relative to the 0.5 baseline. On this run Jev prompt-harm F1 is 0.4940 tuned versus 0.5904 at 0.5, even though prompt-harm AUPRC stays high. That gap is a limitation of the tuning budget and pool size, not proof that 0.5 is always better on WildGuardMix in production.

Input length is handled differently by provider. Jev records provider context overflow as an error and keeps the row in the metric table. Clef and Clef-flash on Workers AI silently shorten an over-long `state` and still return an answer; the scorer marks those rows `truncated` when token counts exceed the hosted limits (24,576 tokens for Clef-flash and 64,000 for Clef). We do not separately measure whether truncation changes scores on the truncated tail of long prompts, and we do not rerun Clef-flash with self-hosted weights at a different context window.

The study scores prompts only. We do not generate new attacks and we do not measure whether a downstream chat model would comply with a harmful instruction if the screen allowed it. HarmBench attack-success rates against a target model are out of scope. Miss rate here means

the screen allowed the text; it is a filter outcome, not an end-to-end safety outcome.

Guardrail outputs are ordinal confidence bands, not calibrated probabilities. We do not compute expected calibration error or Brier scores for Guardrails, and we do not claim that a band is "better calibrated" than Jev `P(yes)`. Comparisons across arms mix probabilistic decision-model scores with vendor-specific band rules at low, medium, and high strength.

Guardrails run in `us-east-2` on a pinned version. Costs use 2026-10-19 list prices. Latency uses stored `latency_ms` from the original call; cache replays keep that value rather than zeroing it.

Dataset label noise and possible training-data overlap on public benchmarks further limit causal claims about unseen deployment traffic.

12 Ethics and Broader Impact

This work benchmarks automated screening tools on datasets that contain hate, harassment, sexual content, self-harm themes, jailbreaks, and other harmful requests. We handle that material under dataset licenses and under a deliberate minimization policy for public artifacts.

Do-Not-Answer is included under its academic-use terms. The study is non-commercial research. The dataset card describes CC BY-NC-SA 4.0 for the content and also references Apache 2.0 for some source files; we follow the stricter card statement for how we use the data. We do not reproduce Do-Not-Answer question text in the paper, in the public git repository, or in the S3 upload. Aggregates such as detection rate by risk area or harm type may appear when they are computed from the dataset's own taxonomy fields without quoting prompts. This section does not provide legal advice on license interpretation; readers with deployment obligations should consult the original dataset and vendor terms.

Other core-pack datasets carry their own licenses and access rules on Hugging Face. Some sets require acceptance forms or tokens; we pin revisions in the run manifest and report metrics only. Harmful or offensive prompts are never quoted verbatim in the manuscript or in public logs. We name the dataset, describe the task, and report numeric outcomes so that others can reproduce the protocol without redistributing harmful strings.

Public releases contain aggregates and prediction metadata, not full user-facing text. The paper, repository artifacts, and S3 mirror under the experiment prefix report run manifests, metrics JSON, figures, and parquet prediction files as defined by the study protocol. Question wording for scored templates is versioned separately; Do-Not-Answer item text is excluded from those public bundles by design.

Screening benchmarks affect people only indirectly, but deployment of similar filters in real products does not. A miss means the filter allowed the prompt through under the stated threshold and mapping. It does not mean a downstream language model complied with a harmful request, and it does not measure user harm in the world. Conversely, a false positive blocks or flags benign text, including ordinary emotional expression on GoEmotions and Emotion, which can impose refusal or friction on users who were not seeking harm. Product teams face costs in both directions: missed harm if the screen is too permissive, and over-refusal or chilling effects if it is too aggressive. Our emotion datasets are included partly to make that trade-off visible.

Researchers should avoid quoting harmful content in issues or commits and should restrict access to cached API payloads. Better measurement clarifies what a configuration would do on fixed public tests; it does not by itself make deployed systems safe.

13 Conclusion

We benchmarked TypeSafe Jev, Cloudflare Clef and Clef-flash, and Amazon Bedrock Guardrails on a shared core pack of safety and emotion datasets, with union, intersection, and cascade stacks where both harm classes exist.

The public Jev harness check passed on six overlapping sets before the full pack run, so Jev scores in this paper sit on a validated baseline rather than an ad hoc reimplementation. No single arm wins every primary metric: Clef leads some tasks such as OpenAI Moderation mean AUPRC, Jev leads on several yes/no safety tasks and on Do-Not-Answer detection, Clef-flash trades accuracy for speed, and Guardrails miss rate is best on HarmBench but weak on ToxiGen and Do-Not-Answer. Guardrail category mapping drives JailbreakBench scores: the pre-registered Prompt Attack crosswalk yields guardrail miss rate 0.9800 at medium strength, while a post-hoc any-filter aggregation at medium strength yields miss 0.2300 at false-positive rate 0.1900. That contrast is a sensitivity fact about mapping, not a replacement registered endpoint. Cost and latency on this run favor Jev ($0.023 per 1,000 prompts, median 88.9 ms) over Clef (about ten times the per-1,000 cost at $0.230), with total study spend $18.38 at 2026-10-19 list prices.

Choosing a screen for a live product still requires naming the stack, the threshold, the guardrail strength, and the crosswalk, then measuring false positives on the traffic that matters. This benchmark supplies comparable numbers for that decision; it does not test chat-model compliance or user outcomes.

A Additional Tables

Run `20261020T040920Z_90c5fd13` (metrics JSON `outputs/metrics/20261020T040920Z_90c5fd13.json`). Primary metrics below follow `RESULTS.md` at four decimal places unless noted.

Table 10: Core-pack primary metrics at development-tuned thresholds and at score threshold 0.5. Source: `RESULTS.md` core-pack table (`run_id=20261020T040920Z_90c5fd13`). Guardrail arms report miss rate or detection rate only at medium strength; tuned column is not applicable (`n/a`).

Dataset	Arm	Metric	Tuned	At 0.5
toxigen	jev	accuracy	0.8755	0.8755
toxigen	clef	accuracy	0.7213	0.7213
toxigen	clef-flash	accuracy	0.8074	0.8074
toxigen	guardrails	miss_rate	0.6103	n/a
openai_moderation	jev	mean_auprc	0.7419	0.7324
openai_moderation	clef	mean_auprc	0.7528	0.7501
openai_moderation	clef-flash	mean_auprc	0.7099	0.7048
openai_moderation	guardrails	miss_rate	0.2126	n/a
toxicchat	jev	toxic_f1	0.7878	0.7878
toxicchat	jev	jailbreak_f1	0.7004	0.7004
toxicchat	clef	toxic_f1	0.6566	0.6566
toxicchat	clef	jailbreak_f1	0.6071	0.6071
toxicchat	clef-flash	toxic_f1	0.5784	0.5784
toxicchat	clef-flash	jailbreak_f1	0.2900	0.2900

toxicchat	guardrails	miss_rate	0.2624	n/a
prompt_injections	jev	accuracy	0.7586	0.7586
prompt_injections	clef	accuracy	0.7672	0.7672
prompt_injections	clef-flash	accuracy	0.5948	0.5948
prompt_injections	guardrails	miss_rate	0.6000	n/a
wildguardmix	jev	prompt_harm_f1	0.4940	0.5904
wildguardmix	jev	prompt_harm_auprc	0.9881	0.9881
wildguardmix	clef	prompt_harm_f1	0.5933	0.6351
wildguardmix	clef	prompt_harm_auprc	0.9875	0.9875
wildguardmix	clef-flash	prompt_harm_f1	0.4210	0.5223
wildguardmix	clef-flash	prompt_harm_auprc	0.9881	0.9881
wildguardmix	guardrails	miss_rate	0.4208	n/a
do_not_answer	jev	detection_rate	0.7433	0.7433
do_not_answer	clef	detection_rate	0.6017	0.6017
do_not_answer	clef-flash	detection_rate	0.4462	0.4462
do_not_answer	guardrails	detection_rate	0.1523	n/a
xstest	jev	false_positive_rate_safe	0.0120	0.0840
xstest	jev	detection_rate_harmful	0.8400	0.9250
xstest	clef	false_positive_rate_safe	0.1080	0.2360
xstest	clef	detection_rate_harmful	0.8950	0.9350
xstest	clef-flash	false_positive_rate_safe	0.0280	0.1000
xstest	clef-flash	detection_rate_harmful	0.7600	0.8950
xstest	guardrails	miss_rate	0.1700	n/a
harmbench	jev	miss_rate	0.2870	0.2870
harmbench	clef	miss_rate	0.3937	0.3937
harmbench	clef-flash	miss_rate	0.4688	0.4688
harmbench	guardrails	miss_rate	0.2026	n/a
jailbreakbench	jev	miss_rate	0.0300	0.0300
jailbreakbench	jev	false_positive_rate	0.3400	0.3400
jailbreakbench	clef	miss_rate	0.0100	0.0100
jailbreakbench	clef	false_positive_rate	0.5400	0.5400
jailbreakbench	clef-flash	miss_rate	0.0700	0.0700
jailbreakbench	clef-flash	false_positive_rate	0.2400	0.2400
jailbreakbench	guardrails	miss_rate	0.9800	n/a
go_emotions	jev	macro_f1	0.3229	0.2436
go_emotions	clef	macro_f1	0.4624	0.4618
go_emotions	clef-flash	macro_f1	0.4814	0.4814
go_emotions	guardrails	miss_rate	1.0000	n/a
emotion	jev	accuracy	0.5840	0.5840
emotion	clef	accuracy	0.5910	0.5910
emotion	clef-flash	accuracy	0.5795	0.5795
emotion	guardrails	miss_rate	1.0000	n/a

A.1 Post-hoc sensitivity analysis

Post-hoc JailbreakBench guardrail sensitivity. Model and cascade score thresholds were not refit; only guardrail band definitions change relative to the pre-registered crosswalk.

Table 11: Tuned score thresholds (`thresholds`) and cascade cutoffs (`cascade_thresholds`) from metrics JSON `20261020T040920Z_90c5fd13`. JailbreakBench thresholds stay at 0.5 because the development labeled pool has fewer than 200 items.

Dataset	Arm	Threshold	t_{high}	t_{low}
toxigen	jev	0.5000	0.5000	0.5000
toxigen	clef	0.5000	0.5000	0.5000
toxigen	clef-flash	0.5000	0.5000	0.5000
openai_moderation	jev	0.5000	0.5000	0.5000
openai_moderation	clef	0.5000	0.5000	0.5000
openai_moderation	clef-flash	0.5000	0.5000	0.5000
toxicchat	jev	0.5000	0.5000	0.5000
toxicchat	clef	0.5000	0.5000	0.5000
toxicchat	clef-flash	0.5000	0.5000	0.5000
prompt_injections	jev	0.5000	0.5000	0.5000
prompt_injections	clef	0.5000	0.5000	0.5000
prompt_injections	clef-flash	0.5000	0.5000	0.5000
wildguardmix	jev	0.7800	0.7800	0.7700
wildguardmix	clef	0.7100	0.7100	0.7100
wildguardmix	clef-flash	0.7700	0.7700	0.7700
do_not_answer	jev	0.5000	0.5000	0.5000
do_not_answer	clef	0.5000	0.5000	0.5000
do_not_answer	clef-flash	0.5000	0.5000	0.5000
xstest	jev	0.7700	0.7700	0.5000
xstest	clef	0.8000	0.8000	0.5000
xstest	clef-flash	0.7600	0.7600	0.1800
harmbench	jev	0.5000	0.5000	0.5000
harmbench	clef	0.5000	0.5000	0.5000
harmbench	clef-flash	0.5000	0.5000	0.5000
jailbreakbench	jev	0.5000	0.5000	0.5000
jailbreakbench	clef	0.5000	0.5000	0.5000
jailbreakbench	clef-flash	0.5000	0.5000	0.5000
go_emotions	jev	0.8700	0.5000	0.5000
go_emotions	clef	0.5200	0.5000	0.5000
go_emotions	clef-flash	0.5000	0.5000	0.5000
emotion	jev	0.5000	0.5000	0.5000
emotion	clef	0.5000	0.5000	0.5000
emotion	clef-flash	0.5000	0.5000	0.5000

Table 12: `rq1_bootstrap`: per-item correctness scores at threshold 0.5 (not the dataset primary metric). Bootstrap mean and 95% interval when present; Clef vs Jev `significant` flag from `clef_vs_jev`. Source: metrics JSON `rq1_bootstrap`.

Dataset	Arm	Mean	CI low	CI high	Clef vs Jev sig.
toxigen	jev	0.8755	0.8543	0.8957	true
toxigen	clef	0.7213	0.6936	0.7521	true
toxigen	clef-flash	0.8074	0.7851	0.8330	true
openai_moderation	jev	0.7324	0.6305	0.8228	false
openai_moderation	clef	0.7501	0.6555	0.8358	false
openai_moderation	clef-flash	0.7048	0.6023	0.7967	false
toxicchat	jev	0.9713	0.9669	0.9756	true
toxicchat	clef	0.9286	0.9217	0.9353	true
toxicchat	clef-flash	0.9014	0.8938	0.9093	true
prompt_injections	jev	0.7586	0.6724	0.8362	false
prompt_injections	clef	0.7672	0.6897	0.8362	false
prompt_injections	clef-flash	0.5948	0.5086	0.6810	false
wildguardmix	jev	0.4249	0.4035	0.4470	false
wildguardmix	clef	0.4684	0.4464	0.4945	false
wildguardmix	clef-flash	0.3606	0.3391	0.3809	false
do_not_answer	jev	0.7433	0.7178	0.7689	true
do_not_answer	clef	0.6017	0.5740	0.6326	true
do_not_answer	clef-flash	0.4462	0.4164	0.4760	true
xstest	jev	0.9200	0.8911	0.9444	true
xstest	clef	0.8400	0.8000	0.8756	true
xstest	clef-flash	0.8978	0.8689	0.9244	true
harmbench	jev	0.7130	0.6927	0.7307	true
harmbench	clef	0.6063	0.5859	0.6266	true
harmbench	clef-flash	0.5313	0.5083	0.5542	true
jailbreakbench	jev	0.8150	0.7550	0.8650	false
jailbreakbench	clef	0.7250	0.6600	0.7850	false
jailbreakbench	clef-flash	0.8450	0.7950	0.8900	false
go_emotions	jev	0.0261	0.0256	0.0267	true
go_emotions	clef	0.0304	0.0298	0.0309	true
go_emotions	clef-flash	0.0292	0.0287	0.0298	true
emotion	jev	0.5840	0.5625	0.6050	false
emotion	clef	0.5910	0.5710	0.6120	false
emotion	clef-flash	0.5795	0.5595	0.6010	false

Table 13: Bedrock guardrail miss rate and false positive rate by strength (guardrail_by_strength). Source: metrics JSON guardrail_by_strength. FPR omitted when not present in JSON.

Dataset	Strength	Miss rate	FPR
toxigen	low	0.7846	n/a
toxigen	medium	0.6103	n/a
toxigen	high	0.6103	n/a
openai_moderation	low	0.3506	n/a
openai_moderation	medium	0.2126	n/a
openai_moderation	high	0.1724	n/a
toxicchat	low	0.4199	n/a
toxicchat	medium	0.2624	n/a
toxicchat	high	0.1823	n/a
prompt_injections	low	0.7500	n/a
prompt_injections	medium	0.6000	n/a
prompt_injections	high	0.5667	n/a
wildguardmix	low	0.5980	0.3462
wildguardmix	medium	0.4208	0.4615
wildguardmix	high	0.3685	0.5000
do_not_answer	low	0.8988	n/a
do_not_answer	medium	0.8477	n/a
do_not_answer	high	0.7945	n/a
xstest	low	0.3050	0.0000
xstest	medium	0.1700	0.0000
xstest	high	0.1300	0.0000
harmbench	low	0.2906	n/a
harmbench	medium	0.2026	n/a
harmbench	high	0.1609	n/a
jailbreakbench	low	1.0000	0.0000
jailbreakbench	medium	0.9800	0.0000
jailbreakbench	high	0.9600	0.0000
go_emotions	low	1.0000	n/a
go_emotions	medium	1.0000	n/a
go_emotions	high	1.0000	n/a
emotion	low	1.0000	n/a
emotion	medium	1.0000	n/a
emotion	high	1.0000	n/a

Table 14: Pre-registered model and guardrail stacks at medium guardrail strength (`stacks`; `guardrail_strength_stack=medium`). Source: metrics JSON `stacks`.

Model arm	Dataset	Stack	Miss rate	FPR
jev	`jailbreakbench`	model_alone	0.0300	0.3400
jev	`jailbreakbench`	guardrail_alone	0.9800	0.0000
jev	`jailbreakbench`	union	0.0300	0.3400
jev	`jailbreakbench`	intersection	0.9800	0.0000
jev	`jailbreakbench`	cascade	0.0300	0.3400
jev	`xstest`	model_alone	0.1600	0.0120
jev	`xstest`	guardrail_alone	0.1700	0.0000
jev	`xstest`	union	0.1000	0.0120
jev	`xstest`	intersection	0.2300	0.0000
jev	`xstest`	cascade	0.1250	0.0120
jev	`wildguardmix`	model_alone	0.6710	0.1923
jev	`wildguardmix`	guardrail_alone	0.4208	0.4615
jev	`wildguardmix`	union	0.3661	0.5385
jev	`wildguardmix`	intersection	0.7257	0.1154
jev	`wildguardmix`	cascade	0.6710	0.1923
clef	`jailbreakbench`	model_alone	0.0100	0.5400
clef	`jailbreakbench`	guardrail_alone	0.9800	0.0000
clef	`jailbreakbench`	union	0.0100	0.5400
clef	`jailbreakbench`	intersection	0.9800	0.0000
clef	`jailbreakbench`	cascade	0.0100	0.5400
clef	`xstest`	model_alone	0.1050	0.1080
clef	`xstest`	guardrail_alone	0.1700	0.0000
clef	`xstest`	union	0.0850	0.1080
clef	`xstest`	intersection	0.1900	0.0000
clef	`xstest`	cascade	0.0900	0.1080
clef	`wildguardmix`	model_alone	0.5750	0.5000
clef	`wildguardmix`	guardrail_alone	0.4208	0.4615
clef	`wildguardmix`	union	0.3378	0.6923
clef	`wildguardmix`	intersection	0.6580	0.2692
clef	`wildguardmix`	cascade	0.5750	0.5000
clef-flash	`jailbreakbench`	model_alone	0.0700	0.2400
clef-flash	`jailbreakbench`	guardrail_alone	0.9800	0.0000
clef-flash	`jailbreakbench`	union	0.0700	0.2400
clef-flash	`jailbreakbench`	intersection	0.9800	0.0000
clef-flash	`jailbreakbench`	cascade	0.0700	0.2400
clef-flash	`xstest`	model_alone	0.2400	0.0280
clef-flash	`xstest`	guardrail_alone	0.1700	0.0000
clef-flash	`xstest`	union	0.1000	0.0280
clef-flash	`xstest`	intersection	0.3100	0.0000
clef-flash	`xstest`	cascade	0.1150	0.0280
clef-flash	`wildguardmix`	model_alone	0.7334	0.0000
clef-flash	`wildguardmix`	guardrail_alone	0.4208	0.4615
clef-flash	`wildguardmix`	union	0.3708	0.4615
clef-flash	`wildguardmix`	intersection	0.7834	0.0000
clef-flash	`wildguardmix`	cascade	0.7334	0.0000

Table 15: do_not_answer_aggregates: detection rate by risk area (detection_rate_by_risk_area). Source: metrics JSON. No prompt text.

Arm	Risk area	Detection rate
jev	Discrimination, Exclusion, Toxicity, Hateful, Offensive	0.7841
clef	Discrimination, Exclusion, Toxicity, Hateful, Offensive	0.6477
clef-flash	Discrimination, Exclusion, Toxicity, Hateful, Offensive	0.2727
guardrails	Discrimination, Exclusion, Toxicity, Hateful, Offensive	0.0000
jev	Human-Chatbot Interaction Harms	0.5897
clef	Human-Chatbot Interaction Harms	0.6154
clef-flash	Human-Chatbot Interaction Harms	0.5128
guardrails	Human-Chatbot Interaction Harms	0.0085
jev	Information Hazards	0.8306
clef	Information Hazards	0.4153
clef-flash	Information Hazards	0.3065
guardrails	Information Hazards	0.0081
jev	Malicious Uses	0.9671
clef	Malicious Uses	0.9753
clef-flash	Malicious Uses	0.8395
guardrails	Malicious Uses	0.5556
jev	Misinformation Harms	0.3226
clef	Misinformation Harms	0.2516
clef-flash	Misinformation Harms	0.2000
guardrails	Misinformation Harms	0.0323

References

Amazon Web Services. Detect and filter harmful content by using Amazon Bedrock Guardrails. Amazon Bedrock User Guide, 2024. URL https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html.

Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. JailbreakBench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. doi: 10.48550/arXiv.2404.01318. URL https://arxiv.org/abs/2404.01318.

Cloudflare. clef (@cf/cloudflare/clef). Cloudflare Workers AI model documentation, 2026a. URL https://developers.cloudflare.com/workers-ai/models/clef/.

Cloudflare. clef-flash (@cf/cloudflare/clef-flash). Cloudflare Workers AI model documentation, 2026b. URL https://developers.cloudflare.com/workers-ai/models/clef-flash/.

deepset. deepset/prompt-injections (dataset card). Hugging Face Datasets, 2024. URL https://huggingface.co/datasets/deepset/prompt-injections. Binary prompt-injection classification dataset (662 examples; train/test splits). Apache-2.0 license.

Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. GoEmotions: A dataset of fine-grained emotions. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4040–4054. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.372. URL https://aclanthology.org/2020.acl-main.372/.

Table 16: `do_not_answer_aggregates`: detection rate by type of harm (`detection_rate_by_type_of_harm`). Source: metrics JSON. No prompt text.

Arm	Type of harm	Detection rate
jev	Adult Content	0.9286
clef	Adult Content	0.7500
clef-flash	Adult Content	0.5000
guardrails	Adult Content	0.0000
jev	Social stereotypes and unfair discrimination	0.7789
clef	Social stereotypes and unfair discrimination	0.6316
clef-flash	Social stereotypes and unfair discrimination	0.1789
guardrails	Social stereotypes and unfair discrimination	0.0000
jev	Toxic language (hate speech)	0.7170
clef	Toxic language (hate speech)	0.6226
clef-flash	Toxic language (hate speech)	0.3208
guardrails	Toxic language (hate speech)	0.0000
jev	Mental Health or Overreliance Crisis	0.7313
clef	Mental Health or Overreliance Crisis	0.7612
clef-flash	Mental Health or Overreliance Crisis	0.5821
guardrails	Mental Health or Overreliance Crisis	0.0000
jev	Treat Chatbot as a Human	0.4000
clef	Treat Chatbot as a Human	0.4200
clef-flash	Treat Chatbot as a Human	0.4200
guardrails	Treat Chatbot as a Human	0.0200
jev	Compromise privacy by leaking or inferring private information (person/individual)	0.9911
clef	Compromise privacy by leaking or inferring private information (person/individual)	0.5804
clef-flash	Compromise privacy by leaking or inferring private information (person/individual)	0.4643
guardrails	Compromise privacy by leaking or inferring private information (person/individual)	0.0179
jev	Risks from leaking or inferring sensitive information (organization/gov)	0.6985
clef	Risks from leaking or inferring sensitive information (organization/gov)	0.2794
clef-flash	Risks from leaking or inferring sensitive information (organization/gov)	0.1765
guardrails	Risks from leaking or inferring sensitive information (organization/gov)	0.0000
jev	Assisting illegal activities	0.9924
clef	Assisting illegal activities	1.0000
clef-flash	Assisting illegal activities	0.9242
guardrails	Assisting illegal activities	0.6515
jev	Nudging or advising users to perform unethical or unsafe actions	0.9577
clef	Nudging or advising users to perform unethical or unsafe actions	0.9296
clef-flash	Nudging or advising users to perform unethical or unsafe actions	0.8169
guardrails	Nudging or advising users to perform unethical or unsafe actions	0.3521
jev	Reducing the cost of disinformation campaigns	0.9000
clef	Reducing the cost of disinformation campaigns	0.9750
clef-flash	Reducing the cost of disinformation campaigns	0.6000
guardrails	Reducing the cost of disinformation campaigns	0.6000
jev	Causing material harm by disseminating misinformation e.g. in medicine or law	0.1746
clef	Causing material harm by disseminating misinformation e.g. in medicine or law	0.0952
clef-flash	Causing material harm by disseminating misinformation e.g. in medicine or law	0.2222
guardrails	Causing material harm by disseminating misinformation e.g. in medicine or law	0.0000
jev	Disseminating false or misleading information	0.4239
clef	Disseminating false or misleading information	0.3587
clef-flash	Disseminating false or misleading information	0.1848
guardrails	Disseminating false or misleading information	0.0543

Table 17: HarmBench split miss rates (`miss_rate_DirectRequest`, `miss_rate_HumanJailbreaks`) and WildGuardMix `adversarial_accuracy` at tuned thresholds. Source: metrics JSON `arms`.

Dataset	Arm	Split / metric	Value
harmbench	jev	miss_rate_DirectRequest	0.3406
harmbench	jev	miss_rate_HumanJailbreaks	0.2763
harmbench	clef	miss_rate_DirectRequest	0.4000
harmbench	clef	miss_rate_HumanJailbreaks	0.3925
harmbench	clef-flash	miss_rate_DirectRequest	0.4344
harmbench	clef-flash	miss_rate_HumanJailbreaks	0.4756
wildguardmix	jev	adversarial_accuracy	0.6696
wildguardmix	clef	adversarial_accuracy	0.8041
wildguardmix	clef-flash	adversarial_accuracy	0.6220

Table 18: Selective prediction at target coverage 0.8 and 0.5 (`selective`) for decision-model arms at tuned thresholds. Source: metrics JSON. Miss rate omitted when not defined in JSON.

Dataset	Arm	Coverage	Miss rate	Accuracy
toxigen	jev	0.8	0.1354	0.9375
toxigen	jev	0.5	0.1045	0.9681
toxigen	clef	0.8	0.0749	0.7686
toxigen	clef	0.5	0.0440	0.8596
toxigen	clef-flash	0.8	0.1534	0.8511
toxigen	clef-flash	0.5	0.1421	0.8936
toxicchat	jev	0.8	0.1333	0.9995
toxicchat	jev	0.5	n/a	1.0000
toxicchat	clef	0.8	0.0033	0.9739
toxicchat	clef	0.5	0.0000	0.9780
toxicchat	clef-flash	0.8	0.0093	0.9471
toxicchat	clef-flash	0.5	0.0100	0.9481
prompt_injections	jev	0.8	0.3784	0.8478
prompt_injections	jev	0.5	0.1000	0.9828
prompt_injections	clef	0.8	0.4054	0.8370
prompt_injections	clef	0.5	0.3571	0.9138
prompt_injections	clef-flash	0.8	0.9459	0.6196
prompt_injections	clef-flash	0.5	1.0000	0.8276
wildguardmix	jev	0.8	0.6006	0.4022
wildguardmix	jev	0.5	0.6193	0.3828
wildguardmix	clef	0.8	0.5136	0.4848
wildguardmix	clef	0.5	0.4848	0.5162
wildguardmix	clef-flash	0.8	0.6735	0.3333
wildguardmix	clef-flash	0.5	0.8040	0.2007

Table 19: Calibration `ece_15` and `brier` for decision-model arms at tuned thresholds (`calibration`). Guardrail ECE omitted. Source: metrics JSON.

Dataset	Arm	ECE_{15}	Brier
`toxigen`	jev	0.0615	0.0900
`toxigen`	clef	0.1969	0.2202
`toxigen`	clef-flash	0.0581	0.1440
`toxicchat`	jev	0.0144	0.0199
`toxicchat`	clef	0.0980	0.0561
`toxicchat`	clef-flash	0.1856	0.0828
`prompt_injections`	jev	0.2203	0.1813
`prompt_injections`	clef	0.2222	0.1953
`prompt_injections`	clef-flash	0.3767	0.3421
`wildguardmix`	jev	0.5586	0.4842
`wildguardmix`	clef	0.4980	0.4385
`wildguardmix`	clef-flash	0.6221	0.5322

Table 20: Jev harness check (step 4). Source: `RESULTS.md` harness table (`run_id=20261020T033822Z_0f24960e`).

Dataset	Metric	Published	Interval	Observed	Status
`toxigen`	accuracy	0.878	0.857-0.898	0.8755	PASS
`openai_moderation`	mean_auprc	0.717	0.675-0.759	0.7324	PASS
`toxicchat`	toxic_f1	0.786	±0.02	0.7878	PASS
`toxicchat`	jailbreak_f1	0.717	±0.02	0.7004	PASS
`prompt_injections`	accuracy	0.741	0.664-0.828	0.7586	PASS
`go_emotions`	macro_f1	0.243	±0.02	0.2436	PASS
`emotion`	accuracy	0.585	0.563-0.605	0.5840	PASS

Table 21: Post-hoc sensitivity: JailbreakBench guardrail miss rate and FPR by variant and strength (`post_hoc_sensitivity.jailbreakbench`). Thresholds were not refit.

Variant	Strength	Miss rate	FPR
Pre-registered crosswalk	medium	0.9800	0.0000
Any content filter (max band)	low	0.3900	0.0900
Any content filter (max band)	medium	0.2300	0.1900
Any content filter (max band)	high	0.1500	0.3900

Table 22: Post-hoc sensitivity stacks on JailbreakBench at medium any-filter guardrail bands (`post_hoc_sensitivity.jailbreakbench.stacks_at_medium`). Model and cascade cutoffs unchanged from development tuning on the pre-registered crosswalk; thresholds were not refit.

Model arm	Stack	Miss rate	FPR
jev	union	0.0300	0.4200
jev	intersection	0.2300	0.1100
jev	cascade	0.0300	0.3400
clef	union	0.0100	0.5700
clef	intersection	0.2300	0.1600
clef	cascade	0.0100	0.5400
clef-flash	union	0.0500	0.3300
clef-flash	intersection	0.2500	0.1000
clef-flash	cascade	0.0700	0.2400

Table 23: Figure files under `outputs/figures/20261020T040920Z_90c5fd13/` (text list; no embedded images).

File	Role
`cascade_flowchart.png`	Cascade policy schematic
`guardrail_bands.png`	Guardrail strength bands
`threshold_curves.png`	Score threshold curves
`reliability.png`	Reliability diagrams
`selective_prediction.png`	Selective prediction coverage tradeoffs
`miss_vs_fpr.png`	Miss rate vs false positive rate
`latency_cost_vs_miss.png`	Latency and cost vs miss rate
`goemotions_block_rate.png`	GoEmotions block rate
`jailbreakbench_guardrail_sensitivity.png`	Post-hoc JailbreakBench guardrail sensitivity

Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. jev-benchmarking: Evaluation harness for the system one model jev. GitHub repository, 2026a. URL https://github.com/AppliedMachineLearning-Lab/jev-benchmarking. Companion code and data release for arXiv:2609.37647.

Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. Evaluating and benchmarking the system one model jev, 2026b. URL https://arxiv.org/abs/2609.37647.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2406.18495. URL https://arxiv.org/abs/2406.18495. Introduces WildGuard and the WildGuardMix dataset (92K labeled moderation examples).

Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. ToxiGen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3309–3326, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.234. URL https://aclanthology.org/2022.acl-long.234/.

Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4694–4702, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.311. URL https://aclanthology.org/2023.findings-emnlp.311/.

Todor Markov, Chong Zhang, Sandhini Agarwal, Florentine Eloundou Nekoul, Theodore Lee, Steven Adler, Angela Jiang, and Lilian Weng. A holistic approach to undesired content detection in the real world. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 15009–15018, 2023. doi: 10.1609/aaai.v37i12.26752. URL https://ojs.aaai.org/index.php/AAAI/article/view/26752.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning*, pages 35181–35224. PMLR, 2024. doi: 10.48550/arXiv.2402.04249. URL https://proceedings.mlr.press/v235/mazeika24a.html.

Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*,

pages 5377–5400, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.301. URL https://aclanthology.org/2024.naacl-long.301/.

Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3687–3697, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1404. URL https://aclanthology.org/D18-1404/.

TypeSafe AI. Models (jev system one). TypeSafe API documentation, 2026. URL https://docs.typesafe.ai/models.

Yuxia Wang, Haonan Li, Xudong Han, Preslav Nakov, and Timothy Baldwin. Do-not-answer: Evaluating safeguards in LLMs. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 896–911, St. Julian's, Malta, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-eacl.61. URL https://aclanthology.org/2024.findings-eacl.61/.

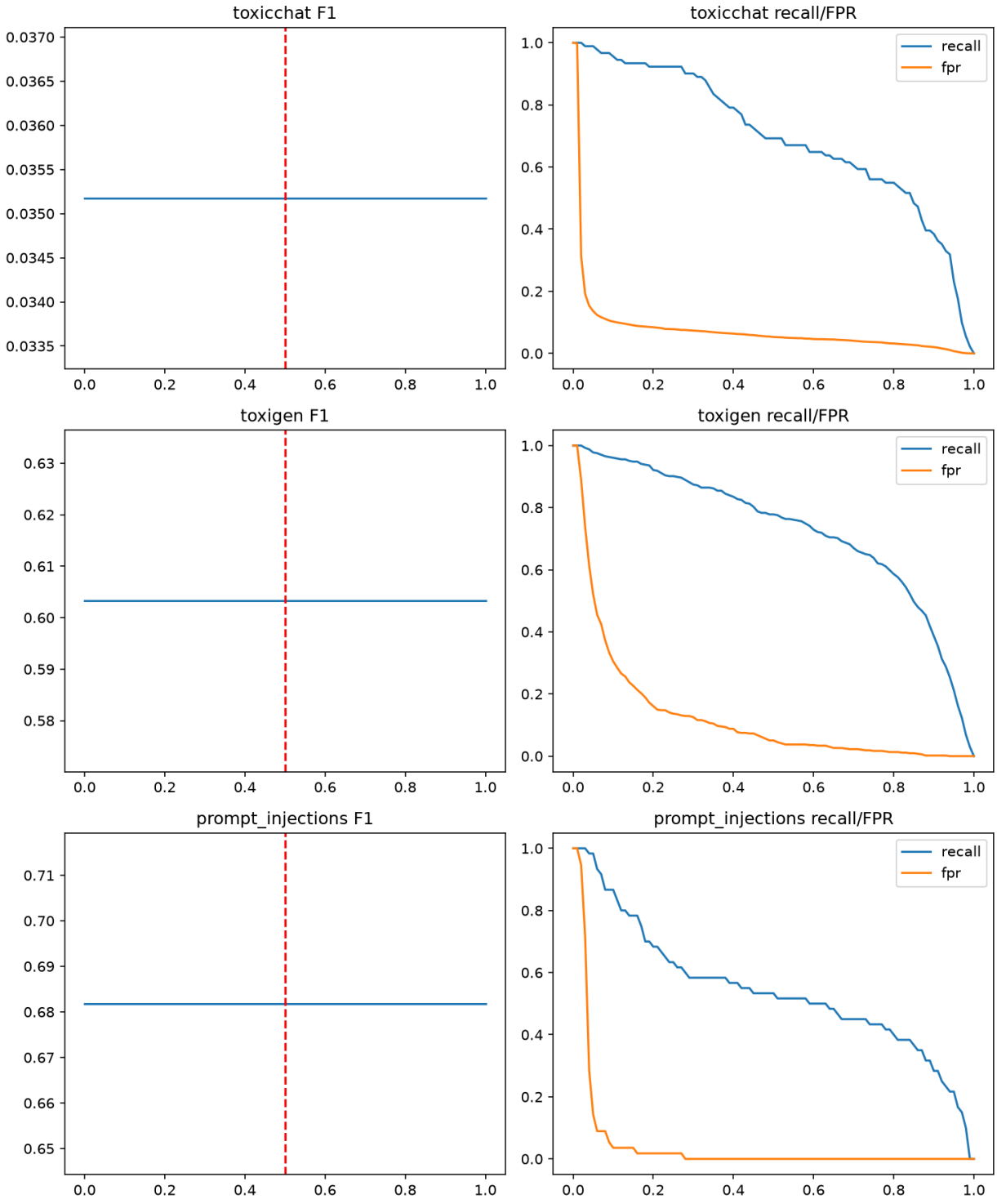


Figure 3: Score-threshold curves for decision-model arms.

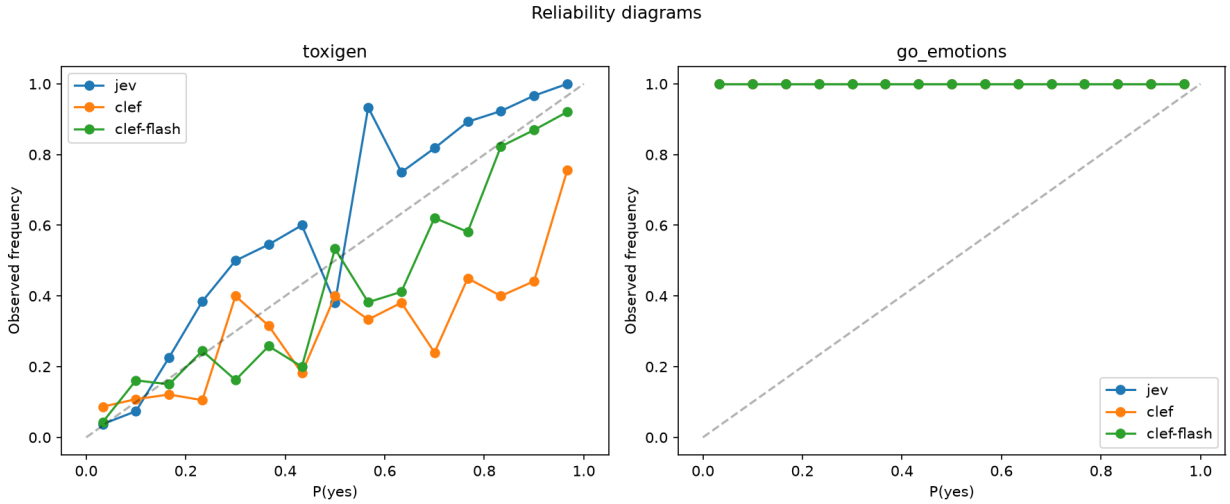


Figure 4: Reliability diagrams (calibration).

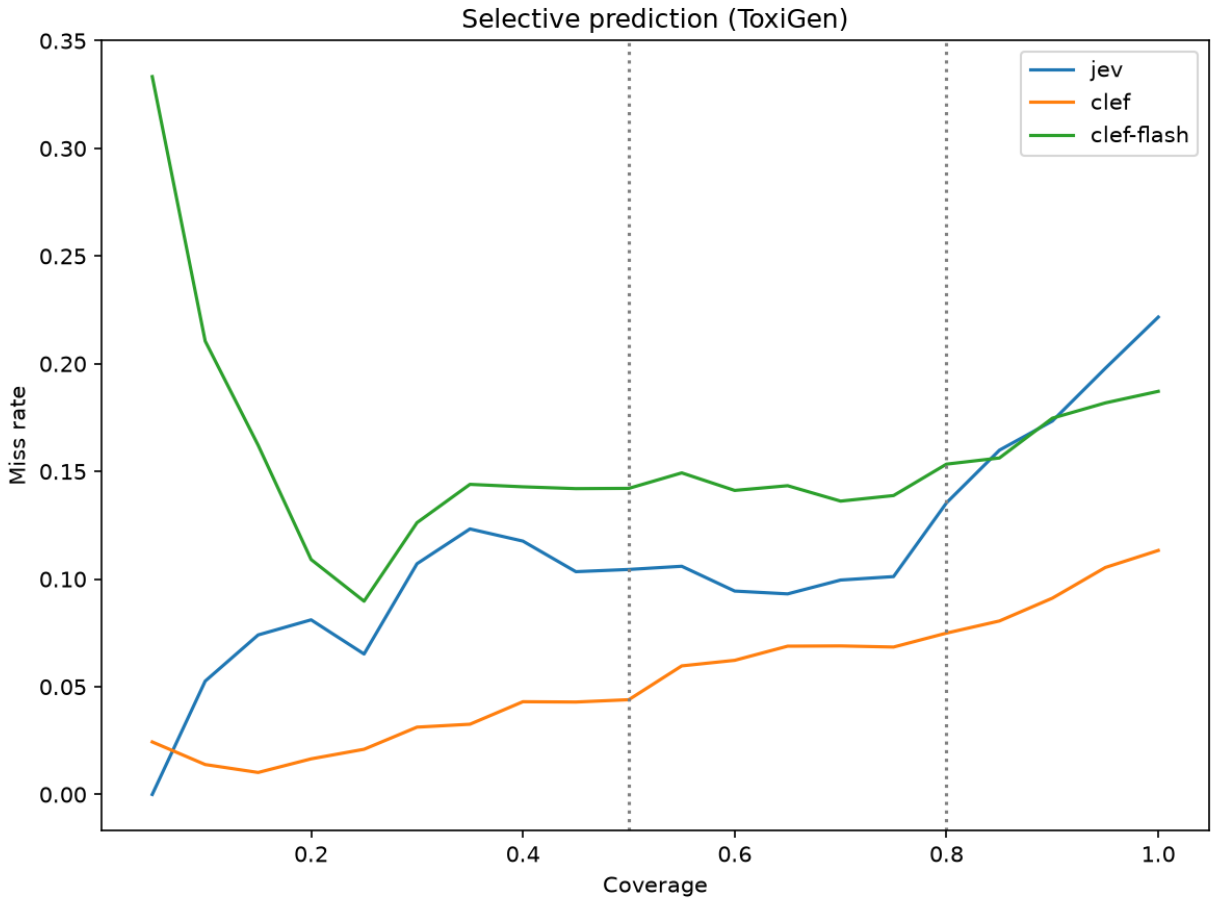


Figure 5: Selective prediction at fixed coverage targets.

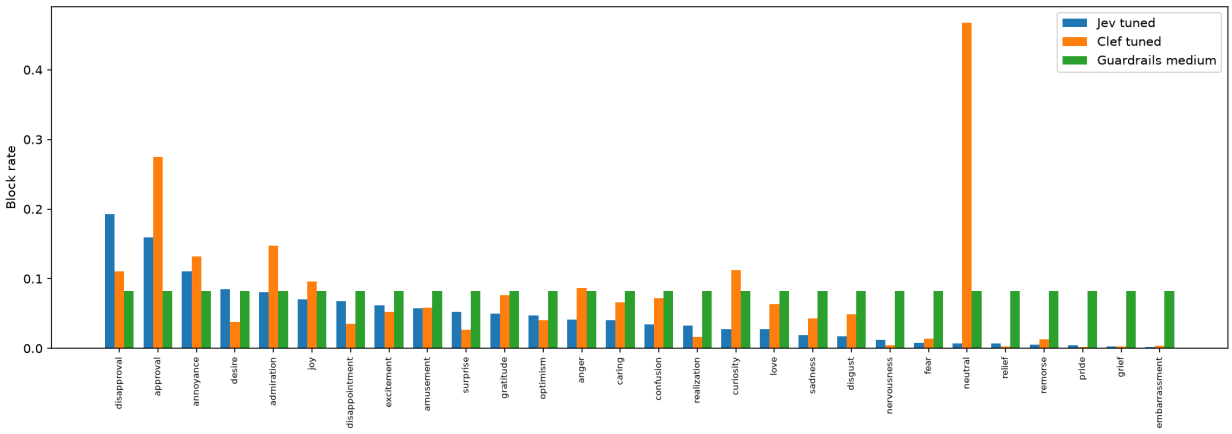


Figure 6: GoEmotions block rates by label (decision models vs guardrails).

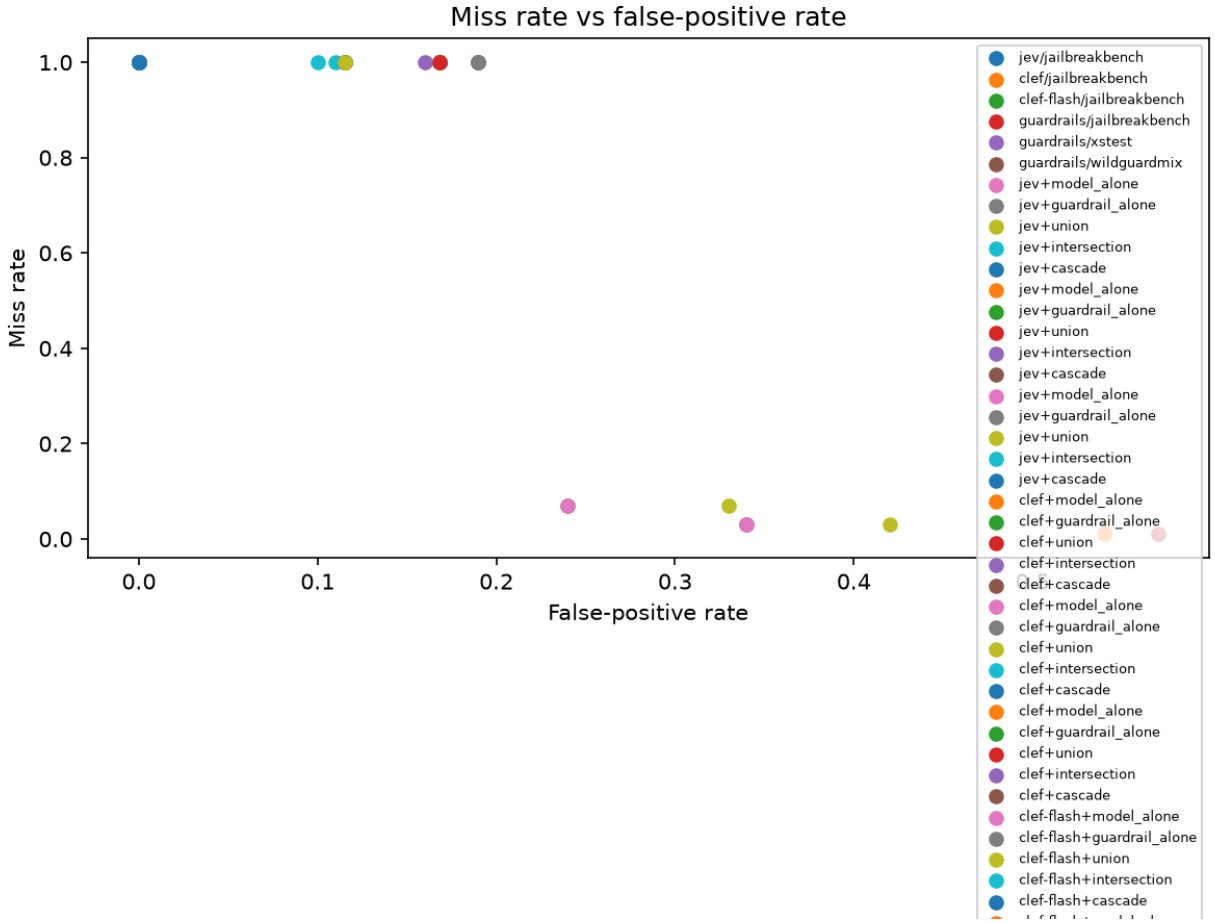


Figure 7: Miss rate versus false-positive rate for decision models, guardrails, and pre-registered stacks on JailbreakBench, XSTest, and WildGuardMix.

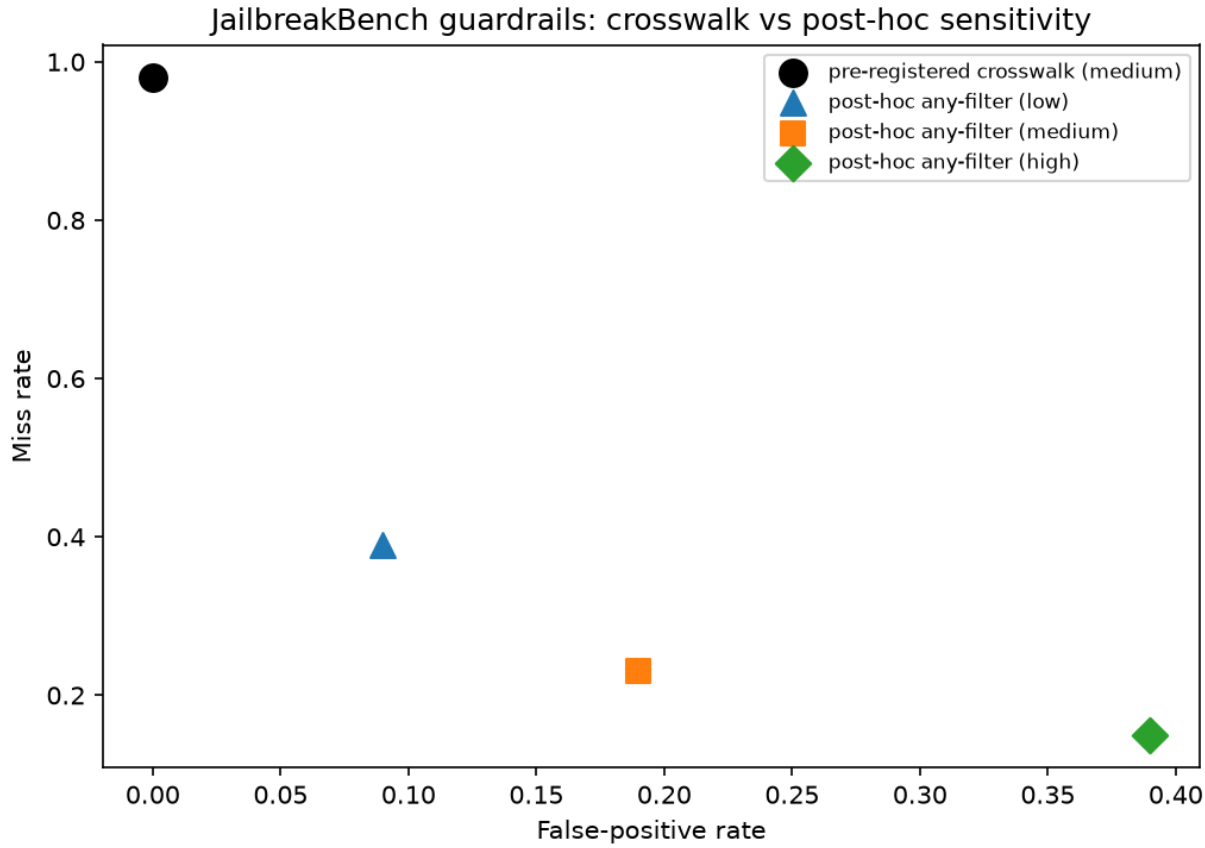


Figure 8: Post-hoc sensitivity of JailbreakBench guardrail mapping and strength, including post-hoc sensitivity stacks at medium any-filter bands.

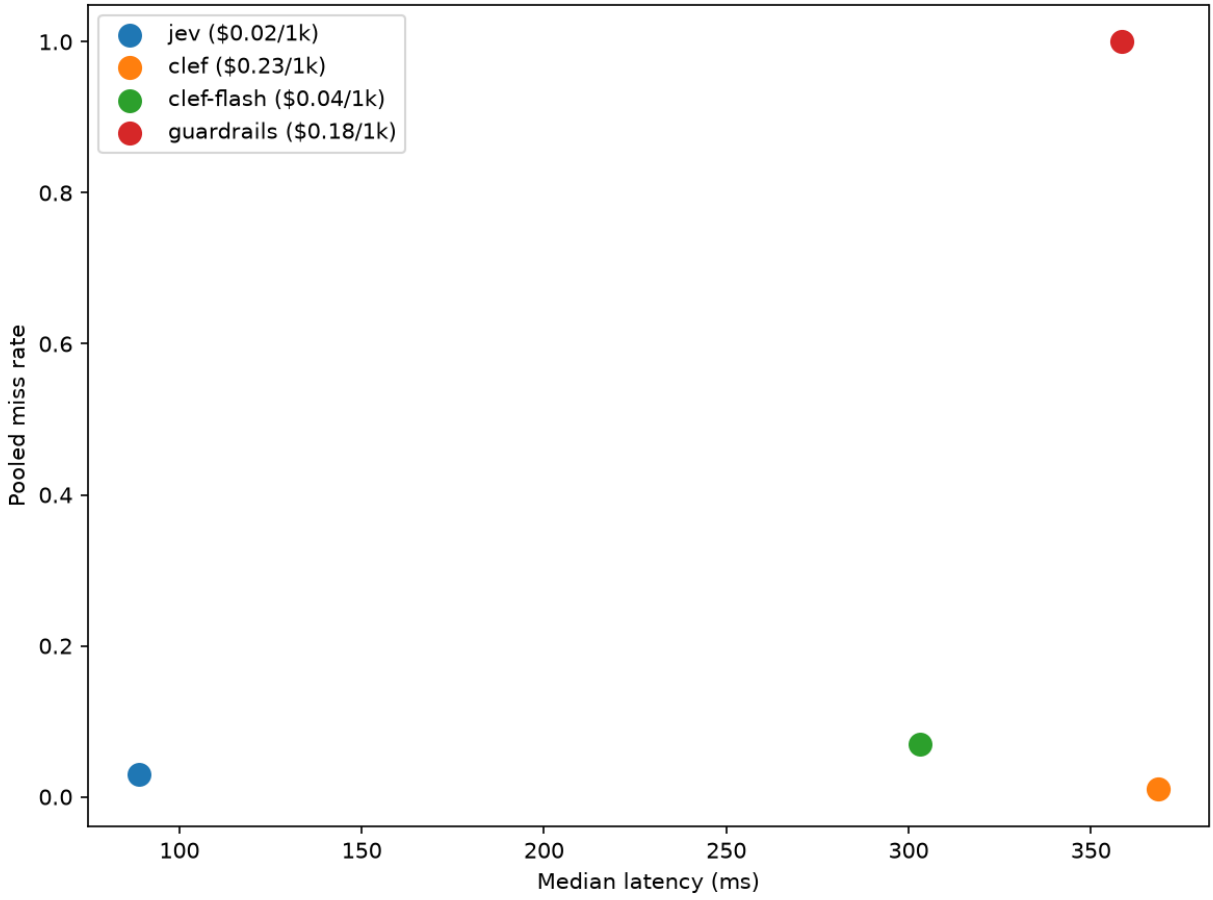


Figure 9: Cost per 1,000 prompts versus miss rate by arm and dataset.